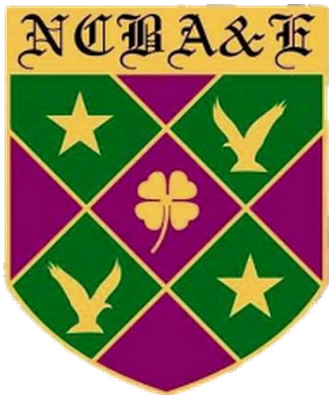



**NATIONAL COLLEGE OF BUSINESS ADMINISTRATION & ECONOMICS**

Front Lane Campus, Lahore

**FINAL YEAR PROJECT REPORT**

# SmartSplit & ExpenseAI

*An AI-Assisted Personal Finance Manager and Group Expense Settlement System using Min-Cash-Flow Optimisation*

Submitted by

**Muhammad Azeem** (BSCSvb-f24152)

**Chanda Safdar** (BSCS-vb-f24-133)

**Saba** (BSITM-f24-25)

Supervised by

**Ms. Sehr Unisa**

A report submitted in partial fulfilment of the requirements for the award of the Bachelor of Science degree (BSCS and BSITM programmes)

**Department of Computer Science**

National College of Business Administration & Economics, Front Lane Campus, Lahore

Session [Session] · November 2026

# DECLARATION

We, **Muhammad Azeem** (Roll No. BSCSvb-f24152), **Chanda Safdar** (Roll No. BSCS-vb-f24-133) and **Saba** (Roll No. BSITM-f24-25), hereby declare that this project report titled "*SmartSplit & ExpenseAI: An AI-Assisted Personal Finance Manager and Group Expense Settlement System using Min-Cash-Flow Optimisation*" is our own work and has not been submitted, in whole or in part, for any other degree or qualification at this or any other institution. All sources of information used in this report have been duly acknowledged and referenced.

_____

**Muhammad Azeem**
Roll No: BSCSvb-f24152

_____

**Chanda Safdar**
Roll No: BSCS-vb-f24-133

_____

**Saba**
Roll No: BSITM-f24-25

Date: _____

# CERTIFICATE OF APPROVAL

It is certified that the project report titled "*SmartSplit & ExpenseAI: An AI-Assisted Personal Finance Manager and Group Expense Settlement System using Min-Cash-Flow Optimisation*" submitted by **Muhammad Azeem** (Roll No. BSCSvb-f24152), **Chanda Safdar** (Roll No. BSCS-vb-f24-133) and **Saba** (Roll No. BSITM-f24-25) has been examined and found satisfactory in partial fulfilment of the requirements for the award of the Bachelor of Science degree (BSCS and BSITM programmes) at the Department of Computer Science, National College of Business Administration & Economics, Front Lane Campus, Lahore.

| | |
|---|---|
| **Ms. Sehr Unisa**<br>Project Supervisor | **[Internal Examiner]**<br>Internal Examiner |
| **[External Examiner]**<br>External Examiner | **[Head of Department]**<br>Department of Computer Science |

# ACKNOWLEDGEMENT

All praise is due to Allah Almighty, who gave us the strength, patience and ability to complete this project.

We are sincerely grateful to our supervisor, Ms. Sehr Unisa, for her continuous guidance, constructive feedback and encouragement at every stage of this work. We also thank the faculty of the Department of Computer Science, National College of Business Administration & Economics, Front Lane Campus, for the knowledge and support provided throughout our degree programmes.

We are thankful to our classmates and friends who tested the application, shared real trip expenses and receipts, and gave honest feedback on its usability. Finally, we owe our deepest gratitude to our parents and families for their prayers, sacrifices and unconditional support.

**Muhammad Azeem** (BSCSvb-f24152)
**Chanda Safdar** (BSCS-vb-f24-133)
**Saba** (BSITM-f24-25)

# ABSTRACT

Managing money is a daily problem for students, young professionals and households in Pakistan. Personal spending is usually tracked in notebooks or not at all, and shared costs on trips or in shared flats lead to long chains of small, confusing repayments. Widely used tools are either designed for foreign markets, keep key features behind paid plans, or treat personal finance and group splitting as two separate products.

This project presents **SmartSplit & ExpenseAI**, a full-stack web application that combines both needs in one account. The personal workspace records income and expenses in Pakistani rupees, applies monthly category budgets, generates recurring bills automatically, and reads printed receipts using a vision language model, with Tesseract.js as an offline fallback. Every extracted line is verified by the user in a human-in-the-loop dialog before it is saved, and verified items are compared against a price book of 100 grocery products to suggest cheaper alternatives. A background job refreshes the price book every morning from five online Pakistani stores, reading only what each store's robots.txt permits and keeping a store's previous prices whenever it cannot be read. Savings goals show how much has been put aside, whether the user is on pace, and how much to save each month, and a zakat calculator compares the user's net wealth with the nisab using gold and silver prices fetched automatically. The group workspace supports multi-payer expenses, four split methods (equal, exact, percentage and line item), a trip currency in which new expenses start, expenses entered in foreign currencies on trips abroad and converted to rupees at daily exchange rates, and a settlement engine based on the greedy Min-Cash-Flow algorithm, which guarantees that a group of V members is settled in at most $V - 1$ payments.

The system is built on the MERN stack (MongoDB, Express, React and Node.js) and exposes 84 REST endpoints. Money is stored as integer rupees, balances are always recomputed from expenses by database aggregation, and every group request is authorised on the server. Authentication uses short-lived JSON Web Tokens with rotating refresh tokens and Google OAuth 2.0, and email can be sent through the Gmail API where the hosting provider blocks SMTP. The implementation is verified by 319 automated unit, API and property-based tests, all passing, and is deployed publicly with every change tested before it goes live. On the demonstration trip of five members and seven expenses, the settlement engine reduced 20 naive pairwise debts to 4 payments.

**Keywords:** expense management, debt simplification, Min-Cash-Flow, greedy algorithm, receipt OCR, vision language model, multi-currency, savings goals, zakat, web scraping, robots.txt, MERN stack, JWT

# TABLE OF CONTENTS

**Declaration** ..... **i**

**Certificate of Approval** ..... **ii**

**Acknowledgement** ..... **iii**

**Abstract** ..... **iv**

**List of Figures** ..... **vii**

**List of Tables** ..... **viii**

**List of Abbreviations** ..... **ix**

**Chapter 1: Introduction** ..... **1**

1.1 Background ..... 1

1.2 Problem Statement ..... 1

1.3 Objectives ..... 2

1.4 Scope ..... 2

1.5 Significance of the Project ..... 3

1.6 Development Methodology ..... 3

1.7 Organisation of the Report ..... 4

**Chapter 2: Literature Review** ..... **5**

2.1 Personal Finance Management Applications ..... 5

2.2 Group Expense Splitting Applications ..... 5

2.3 Debt Simplification Algorithms ..... 5

2.4 Receipt Text Extraction (OCR) ..... 6

2.5 String Similarity for Product Matching ..... 6

2.6 Collecting Prices from Retailer Websites ..... 6

2.7 Web Application Security ..... 6

2.8 Comparison with Existing Systems ..... 7

2.9 Research Gap ..... 7

**Chapter 3: Requirements Analysis** ..... **8**

3.1 Users and Stakeholders ..... 8

3.2 Functional Requirements ..... 8

3.3 Non-Functional Requirements ..... 10

3.4 Use Case Diagrams ..... 11

3.5 Use Case Descriptions ..... 13

3.6 Business Rules ..... 14

3.7 Hardware and Software Requirements ..... 15

**Chapter 4: System Design** ..... **16**

4.1 System Architecture ..... 16

4.2 Database Design ..... 17

4.3 Expense Split Calculation ..... 18

4.4 Settlement Algorithm (Min-Cash-Flow) ..... 19

4.5 Receipt Processing Pipeline ..... 22

4.6 Price Matching and Savings Suggestions ..... 23

4.7 Multi-Currency Conversion 23
4.8 Savings Goal Progress 24
4.9 Security Design 24
4.10 API Design 25
4.11 User Interface Design 26

**Chapter 5: Implementation 27**
5.1 Technology Stack 27
5.2 Project Structure 28
5.3 Backend Implementation 28
5.4 Frontend Implementation 34
5.5 User Interface 35
5.6 Deployment 45

**Chapter 6: Testing and Evaluation 47**
6.1 Testing Strategy 47
6.2 Test Suite Summary 47
6.3 Selected Test Cases 48
6.4 Property-Based Testing of Settlement 50
6.5 Security Testing 50
6.6 Defects Found and Fixed 51
6.7 Evaluation 52

**Chapter 7: Conclusion and Future Work 53**
7.1 Conclusion 53
7.2 Limitations 53
7.3 Future Work 54

**References 55**

**Appendix A: API Endpoint Reference 57**

**Appendix B: User Manual 63**

**Appendix C: Installation Guide 64**

# LIST OF FIGURES

Figure 1.1: Incremental development phases of the project ..... 3
Figure 3.1: Use case diagram — account and personal workspace ..... 12
Figure 3.2: Use case diagram — group workspace ..... 12
Figure 4.1: Three-tier architecture of SmartSplit & ExpenseAI ..... 16
Figure 4.2: Entity relationship diagram ..... 17
Figure 4.3: Flowchart of the Min-Cash-Flow settlement algorithm ..... 20
Figure 4.4: Sequence diagram of receipt scanning and verification ..... 22
Figure 4.5: Sequence diagram of sign-in, token refresh and reuse detection ..... 25
Figure 5.1: Sign-in page ..... 35
Figure 5.2: Dashboard combining both workspaces ..... 35
Figure 5.3: Groups list with total spent and personal balance ..... 36
Figure 5.4: Group expenses of the Murree Trip ..... 36
Figure 5.5: Group balances, which always net to zero ..... 37
Figure 5.6: Add Expense form with multi-payer rows and split methods ..... 37
Figure 5.7: Expense entered in UAE dirhams with the live rupee preview ..... 38
Figure 5.8: Dubai Trip expenses: rupee amounts with the original AED figures ..... 39
Figure 5.9: Settle Up after optimisation: 20 naive debts reduced to 4 payments ..... 39
Figure 5.10: Personal workspace with budget bands and transaction form ..... 40
Figure 5.11: Savings goals with progress, pace and the monthly amount needed ..... 40
Figure 5.12: Zakat calculator with pre-filled figures, automatically fetched metal prices and the result ..... 41
Figure 5.13: Receipt scanning page with recent scans and savings found ..... 42
Figure 5.14: Human-in-the-loop verification dialog (demonstration data uses a placeholder image) ..... 42
Figure 5.15: Price book of benchmark prices, maintained by super administrators ..... 43
Figure 5.16: Live store prices panel on the Admin page after a refresh in which Chase Up failed and kept its previous prices ..... 43
Figure 5.17: Personal reports: category distribution, inflow vs outflow, budget vs actual ..... 44
Figure 5.18: Group report: paid vs consumed per member and category split ..... 45

# LIST OF TABLES

Table 2.1: Comparison of the proposed system with existing tools ..... 7
Table 3.1: User roles of the system ..... 8
Table 3.2: Functional requirements ..... 8
Table 3.3: Non-functional requirements ..... 10
Table 3.4: Use case UC-01: Add a shared expense ..... 13
Table 3.5: Use case UC-02: Settle up a group ..... 13
Table 3.6: Use case UC-03: Scan and verify a receipt ..... 13
Table 3.7: Business rules enforced by the system ..... 14
Table 3.8: Hardware and software requirements ..... 15
Table 4.1: Collections and their purpose ..... 17
Table 4.2: Net balances of the Murree Trip group ..... 21
Table 4.3: Iterations of Min-Cash-Flow on the Murree Trip group ..... 21
Table 4.4: Conversion of AED 412.50 shared equally by four members at 1 AED = Rs 75.2955 ..... 24
Table 4.5: Token design ..... 24
Table 5.1: Technologies used ..... 27
Table 5.2: Online stores read by the price refresh ..... 31
Table 6.1: Automated test results by file ..... 47
Table 6.2: Selected test cases ..... 48
Table 6.3: Security review findings and fixes ..... 51
Table 6.4: Notable defects found during testing ..... 51
Table 6.5: Evaluation against objectives ..... 52
Table A.1: REST API endpoints ..... 57
Table B.1: Step-by-step guide to the main tasks ..... 63

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| API | Application Programming Interface |
| CI/CD | Continuous Integration / Continuous Deployment |
| CORS | Cross-Origin Resource Sharing |
| CSV | Comma-Separated Values |
| ERD | Entity Relationship Diagram |
| FR | Functional Requirement |
| FX | Foreign Exchange |
| FYP | Final Year Project |
| HTTP(S) | Hypertext Transfer Protocol (Secure) |
| JSON | JavaScript Object Notation |
| JSON-LD | JSON for Linking Data |
| JWT | JSON Web Token |
| LLM | Large Language Model |
| MCF | Min-Cash-Flow |
| MERN | MongoDB, Express, React, Node.js |
| NFR | Non-Functional Requirement |
| OAuth | Open Authorization |
| OCR | Optical Character Recognition |
| ODM | Object Document Mapper |
| OWASP | Open Worldwide Application Security Project |
| PDF | Portable Document Format |
| PKR | Pakistani Rupee |
| REST | Representational State Transfer |
| RFC | Request for Comments |
| SMTP | Simple Mail Transfer Protocol |
| SPA | Single-Page Application |
| SSO | Single Sign-On |
| TTL | Time To Live |
| UI | User Interface |
| UTC | Coordinated Universal Time |
| XLSX | Office Open XML Spreadsheet |
| XML | Extensible Markup Language |
| XSS | Cross-Site Scripting |

# CHAPTER 1
# INTRODUCTION

This chapter introduces the problem domain, states the problem addressed by the project, and sets out its objectives, scope, significance and development methodology. It closes with an outline of the remaining chapters.

## 1.1 Background

Personal financial management is the routine of recording income and expenditure, comparing it with a plan, and adjusting behaviour. Research and everyday experience agree that people who track their spending make better financial decisions, yet most people find manual tracking tedious and stop after a few weeks. Receipts are lost, small cash purchases are forgotten, and categories are applied inconsistently.

A second, closely related problem appears whenever costs are shared. University trips, shared flats, weddings and office events produce many bills paid by different people for different subsets of the group. At the end, each participant needs to know how much they owe or are owed, and the group needs a practical way to settle. Settling every bill separately produces many small transfers; with mobile wallets such as JazzCash and Easypaisa and the State Bank of Pakistan's instant payment system Raast [19], each transfer is easy, but the number of transfers and the confusion about who has already paid remain the real cost.

Recent advances make a better solution practical. Vision language models can read photographs of printed receipts and return structured data, and established algorithms can reduce a web of debts to a small set of payments [1]. Modern web frameworks allow both capabilities to be delivered in a responsive application that works on any phone or laptop browser without installation.

## 1.2 Problem Statement

Existing tools address personal tracking and group splitting separately, require manual entry of every item, rarely support Pakistani payment methods or price references, and do not explain how settlement payments are derived. As a result, users must maintain two systems, re-type receipt contents by hand, and settle group debts through more transfers than necessary.

The project therefore addresses the following question: *how can a single, secure web application help users record personal and shared expenses with minimal manual effort, guarantee that every rupee is accounted for exactly, and settle group debts with the fewest practical number of payments?*

## 1.3 Objectives

The objectives of the project are to:

1. Design and implement a personal workspace for recording income and expenses in PKR with categories, local payment methods, monthly budgets and automatically generated recurring entries.
2. Implement AI-assisted receipt scanning that extracts line items from a photo using a vision language model, falls back to offline OCR, and requires human verification before saving.
3. Suggest savings by matching verified receipt items against a local benchmark of market prices, refreshed automatically from online stores within the limits each store sets in its robots.txt.
4. Implement a group workspace with role-based membership, multi-payer expenses and four split methods that always sum exactly to the bill total.
5. Implement a settlement engine based on the greedy Min-Cash-Flow algorithm that settles a group of V members in at most V − 1 payments, and demonstrate the reduction compared with naive repayment.
6. Help users save towards personal goals and estimate the zakat due on their wealth, and let groups record trips abroad in foreign currencies, starting in the trip's own currency, while all accounting stays in rupees.
7. Provide reports, charts and CSV, Excel and PDF exports for both workspaces.
8. Secure the system with industry-standard authentication, authorisation, input validation and rate limiting.
9. Verify correctness through automated unit, API and property-based tests.
10. Deploy the system publicly so that every change is tested automatically before it goes live.

## 1.4 Scope

**In scope.** A responsive web application for individual users and small groups (trips, flats and events of roughly 2 to 20 members); amounts in Pakistani rupees, with group expenses also accepted in 15 foreign currencies and converted to rupees; savings goals; a zakat estimate; email/password and Google sign-in; receipt photos in JPG or PNG format up to 8 MB; a price book of about 100 grocery products with indicative prices for seven stores, refreshed daily with live prices from five online stores; deployment on free cloud tiers.

**Out of scope.** Moving money between accounts (users pay through their own wallet or bank and then mark the payment as paid), bank account synchronisation, balances kept in a currency other than the rupee, native mobile applications, reading prices from any store or

path that its robots.txt disallows, religious rulings on disputed zakat questions (the calculator gives an estimate), and income tax or investment features.

## 1.5 Significance of the Project

- **For users:** one account covers personal and shared money; receipts are captured in seconds; groups settle with the minimum practical number of transfers.
- **For the local context:** PKR formatting, JazzCash, Easypaisa and Raast as first-class payment methods, prices from Pakistani retailers, and a zakat calculator that works in tola and rupees.
- **Academically:** the project applies a graph-based greedy algorithm with a provable bound, integer arithmetic for financial correctness, human-in-the-loop AI, fuzzy string matching and secure token-based authentication in a single coherent system.

## 1.6 Development Methodology

The system was developed using the **incremental model** [20]. The work was divided into ten increments (Phases 0 to 9), each delivering a usable slice of functionality. Every increment followed the same cycle: refine the requirements, implement, write automated tests, and record a phase report before the next increment began. Core money logic (split calculation and settlement) was written test-first. Figure 1.1 shows the increments.

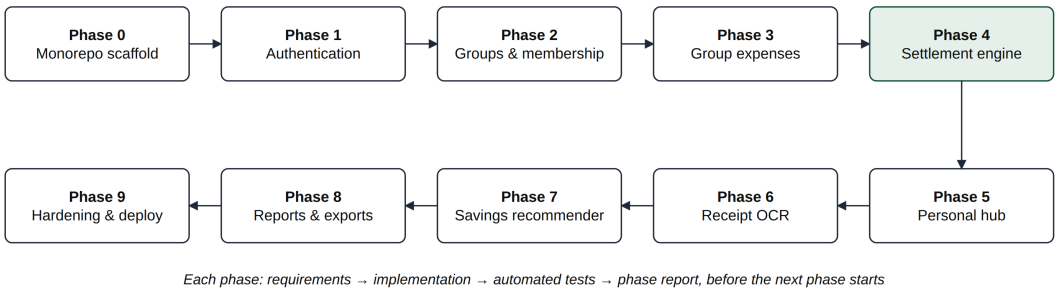



**Figure 1.1:** Incremental development phases of the project

The incremental model was preferred over the waterfall model because later features (OCR, savings suggestions and reports) depend on earlier ones (authentication, groups and expenses), and delivering each layer fully tested reduced the risk of defects compounding across the system. After the ten planned increments, further increments based on user and supervisor feedback added an invitation inbox, member roles, a price book with super-administrators, public deployment, multi-currency trip expenses and savings goals, and finally a trip currency for groups, a zakat calculator, live store prices and email through the Gmail API.

## 1.7 Organisation of the Report

Chapter 2 reviews existing systems and the relevant literature. Chapter 3 specifies the requirements. Chapter 4 presents the system design, including architecture, database and algorithms. Chapter 5 describes the implementation and user interface. Chapter 6 reports testing and evaluation. Chapter 7 concludes and outlines future work. Appendices provide the API reference, a user manual and an installation guide.

# CHAPTER 2
# LITERATURE REVIEW

This chapter reviews existing applications for personal and group expense management, the algorithmic background of debt simplification, techniques for receipt text extraction and product matching, the collection of prices from retailer websites, and the security standards used by modern web applications. It concludes by identifying the gap addressed by this project.

## 2.1 Personal Finance Management Applications

Personal finance applications such as Wallet by BudgetBakers [17] and many similar budgeting apps allow users to record transactions, group them into categories and compare them against budgets. Spreadsheets remain the most common free alternative. These tools are effective for disciplined users, but entry is largely manual and they are not designed for splitting costs with other people. Support for Pakistani payment methods and local price references is limited.

## 2.2 Group Expense Splitting Applications

Splitwise [15] and Tricount [16] are widely used for shared expenses. Both record who paid for what, compute balances and propose repayments; Splitwise offers a "simplify debts" option that reduces the number of repayments. Advanced features such as itemised receipt entry are offered in paid tiers in some products. These applications focus on the group and do not provide a full personal budgeting workspace, so users who want both must use two separate systems.

## 2.3 Debt Simplification Algorithms

A set of shared expenses can be modelled as a directed weighted graph in which an edge from A to B of weight w means that A owes B an amount w. Only each person's **net balance** (total paid minus total owed) matters for settlement: any set of payments that brings every net balance to zero is a valid settlement. Verhoeff [1] analyses this problem in detail and shows that finding the absolute minimum number of payments is computationally hard, since it requires partitioning the members into the largest possible number of zero-sum subgroups, which is related to the NP-complete subset-sum problem [2].

The practical alternative is a **greedy** strategy: repeatedly match the largest creditor with the largest debtor and transfer the smaller of the two amounts. Each transfer clears at least one person, so a group of V members needs at most $V - 1$ payments [1], [3]. This bound is tight

in general and the method runs in polynomial time, which makes it the standard choice in practice. This project adopts the greedy Min-Cash-Flow method and reports its result against the naive pairwise approach.

## 2.4 Receipt Text Extraction (OCR)

Traditional OCR engines such as Tesseract [4] segment an image into lines and characters and recognise them using trained models. They work offline and at no cost, but return unstructured text; a separate rule-based step is needed to identify item names, quantities and prices, and accuracy drops on crumpled, faded or photographed receipts. Vision language models accept an image directly and can be instructed to return a fixed JSON structure, which removes the need for hand-written parsing rules and handles varied layouts better [14]. Because any automated extraction can misread digits, financial applications should keep a human in the loop to confirm results before they affect stored balances.

## 2.5 String Similarity for Product Matching

Receipt item names are abbreviated and inconsistent ("COOKING OIL 3LTR", "Cooking Oil 3 Litre"). The Sørensen–Dice coefficient [5] measures the similarity of two strings as twice the number of shared character bigrams divided by the total number of bigrams, giving a score between 0 and 1. It is simple, fast and robust to small differences in word order and spelling, which makes it suitable for matching receipt text against a product catalogue after normalisation.

## 2.6 Collecting Prices from Retailer Websites

Retail websites publish their prices in forms intended for machines as well as people. The Robots Exclusion Protocol [23] lets a site list, in a `robots.txt` file at its root, the paths that automated clients may and may not fetch; when several rules match a path, the longest one decides. A well-behaved collector reads this file before it uses any path, identifies itself honestly and keeps its request rate low. Many stores also list every product page in an XML sitemap and embed a schema.org `Product` description [24] in each page as JSON-LD for search engines, giving the product name, price and availability in a fixed structure. Reading these structured sources is more dependable than interpreting the visual layout of a page, which changes far more often.

## 2.7 Web Application Security

The OWASP Top 10 [9] lists the most critical web application risks, including broken access control, cryptographic failures, injection and identification and authentication failures. JSON Web Tokens [6] allow a server to issue signed, self-contained access tokens; keeping them short-lived and pairing them with rotating refresh tokens limits the damage of a stolen token.

OAuth 2.0 [7] enables sign-in through an identity provider such as Google without the application handling the user's password. Passwords that are stored must be hashed with a deliberately slow, salted algorithm such as bcrypt [8]. For financial values, binary floating-point arithmetic introduces representation errors [10], so amounts should be stored as integers of the smallest currency unit.

## 2.8 Comparison with Existing Systems

Table 2.1 compares the proposed system with representative existing tools, based on their publicly documented features at the time of writing.

**Table 2.1:** Comparison of the proposed system with existing tools

| Feature | Spreadsheet | Wallet | Splitwise | Tricount | Proposed |
|---|---|---|---|---|---|
| Personal income/expense tracking | Manual | Yes | Limited | No | Yes |
| Monthly category budgets | Manual | Yes | No | No | Yes |
| Group expense splitting | Manual | No | Yes | Yes | Yes |
| Debt simplification | No | No | Yes | Yes | Yes, with $V-1$ bound shown |
| Itemised receipt OCR | No | Limited | Paid tier | Limited | Yes, human-verified |
| Price comparison / savings tips | No | No | No | No | Yes |
| JazzCash / Easypaisa / Raast | Manual | Limited | No | No | Yes |
| Personal and group in one account | Manual | No | No | No | Yes |

## 2.9 Research Gap

The review shows that no single, free tool combines personal budgeting, AI-assisted and human-verified receipt capture, local price comparison and group settlement with a stated payment bound, for the Pakistani context. SmartSplit & ExpenseAI is designed to fill this gap, while applying established results from algorithms, OCR, string matching and security engineering.

# CHAPTER 3
# REQUIREMENTS ANALYSIS

This chapter identifies the users of the system and specifies its functional and non-functional requirements, use cases, business rules, and hardware and software requirements.

## 3.1 Users and Stakeholders

**Table 3.1:** User roles of the system

| Role | Description |
|---|---|
| Guest | An unauthenticated visitor who can register, sign in (email or Google) and reset a forgotten password. |
| Registered user | Owns a private personal workspace (transactions, budgets, receipts, reports) and may belong to any number of groups. |
| Group member | An active member of a group who can view its expenses and balances, add expenses, preview a settlement and mark payments as paid. |
| Group admin | A member with management rights: edit group settings, add, invite or remove members, approve join requests, promote or demote members, rotate the share code and record settlements. The creator of a group is its first admin. |
| Super admin | A person whose email is listed in the ADMIN_EMAILS server setting. Sees the Admin page (totals, newest users and groups, service checklist and live store prices), maintains the benchmark price book, can start a store-price refresh and sets the bazaar gold and silver prices used for zakat. |
| External services | Vision AI (receipt extraction), Cloudinary (image storage), Google OAuth (sign-in), ExchangeRate-API (exchange rates), Gold API (gold and silver prices), five online stores (prices), the Gmail API or Brevo SMTP (email) and the daily scheduler. |

## 3.2 Functional Requirements

**Table 3.2:** Functional requirements

| ID | Module | Requirement |
|---|---|---|
| FR-01 | Accounts | The system shall allow users to register and sign in with email and password. |
| FR-02 | Accounts | The system shall allow users to sign in with Google OAuth 2.0 and link it to an existing account with the same email. |
| FR-03 | Accounts | The system shall provide password reset through a single-use emailed link that expires after 30 minutes. |
| FR-04 | Accounts | The system shall allow users to update their profile and change their password. |
| FR-05 | Personal | The system shall allow users to record money received (credit) and spent (debit) with title, amount, date, category, payment method and note. |

| ID | Module | Requirement |
|---|---|---|
| FR-06 | Personal | The system shall provide seven built-in categories (Food, Utilities, Travel, Shopping, Healthcare, Housing, Misc) and allow user-defined categories. |
| FR-07 | Personal | The system shall support the payment methods Cash, Bank Transfer, Card, JazzCash, Easypaisa and Raast. |
| FR-08 | Personal | The system shall allow a monthly budget cap per category and show consumption as green (< 80%), amber (80–99%) or red (≥ 100%). |
| FR-09 | Personal | The system shall generate weekly or monthly recurring transactions automatically, including any periods missed while the server was offline. |
| FR-10 | Personal | The system shall show a monthly summary: received, spent, remaining, comparison with the previous month and spending per category. |
| FR-11 | Receipts | The system shall accept a receipt photo (JPG/PNG, up to 8 MB) and process it in the background. |
| FR-12 | Receipts | The system shall extract store, date, line items and total using a vision AI model, and fall back to offline OCR if the model is unavailable. |
| FR-13 | Receipts | The system shall present extracted lines for user verification and editing, and warn when the lines and the printed total differ by more than 5%. |
| FR-14 | Receipts | The system shall save a verified receipt as a personal transaction or as a pre-filled line-item group expense. |
| FR-15 | Savings | The system shall compare verified items with benchmark prices and show cheaper alternatives and the total potential saving. |
| FR-16 | Groups | The system shall allow users to create groups (trip, home, event, other) and become their admin. |
| FR-17 | Groups | The system shall support joining a group by direct add, single-use email invite link, or share code with admin approval, and shall show pending invitations in an inbox where they can be accepted or declined. |
| FR-18 | Groups | The system shall record group expenses paid by one or more members. |
| FR-19 | Groups | The system shall split expenses equally, by exact amounts, by percentage, or by line item, with live validation messages. |
| FR-20 | Groups | The system shall display each member's net balance, recomputed from all expenses and paid settlements. |
| FR-21 | Settlement | The system shall preview the optimised payment list without saving, showing naive debt count versus optimised payment count. |
| FR-22 | Settlement | The system shall let a group admin record a settlement, mark the group as settled, and keep an audit log entry. |
| FR-23 | Settlement | The system shall allow each payment to be marked as paid, updating balances immediately. |
| FR-24 | Reports | The system shall provide monthly and six-month personal reports and group summary reports with charts. |
| FR-25 | Reports | The system shall export transactions and group reports as CSV, Excel (XLSX) and PDF. |

| ID | Module | Requirement |
|---|---|---|
| FR-26 | Notifications | The system shall notify group members when an expense is added, a member joins, a settlement is created or a payment is marked paid. |
| FR-27 | Groups | The system shall accept a group expense in any of 16 supported currencies, convert it to whole rupees at the current exchange rate, and keep the original amounts and rate for display. |
| FR-28 | Goals | The system shall let users create savings goals with a target amount and date, record deposits and withdrawals, and show progress, whether they are on track and the amount to save each month. |
| FR-29 | Groups | The system shall let invite links and share links be sent through WhatsApp, and let an admin promote or demote members while always keeping at least one admin. |
| FR-30 | Admin | The system shall let super admins maintain the benchmark price book (add, edit, delete, Excel or CSV import and Excel export) and view an overview of users and groups. |
| FR-31 | Groups | The system shall let a group be given a trip currency from the 16 supported currencies, start every new expense in that group in it, and keep all balances and settlements in rupees. |
| FR-32 | Zakat | The system shall estimate zakat as 2.5% of the user's net wealth when it reaches the nisab of 52.5 tola of silver or 7.5 tola of gold, pre-fill the figures it already holds, and record the zakat as a personal transaction only when the user confirms. |
| FR-33 | Zakat | The system shall fetch international gold and silver prices automatically, convert them to rupees per tola, let a super admin save bazaar prices that are used until the next refresh, and let any user type their own prices for one calculation. |
| FR-34 | Prices | The system shall refresh the price book every day from online stores whose robots.txt permits it, keep the previous prices of any store that cannot be read, and show super admins the result of each run with an option to start one at once. |
| FR-35 | Email | The system shall send password-reset and invitation emails through the Gmail API when it is configured and otherwise through SMTP, and shall offer the link for sharing when an email cannot be sent. |

## 3.3 Non-Functional Requirements

**Table 3.3:** Non-functional requirements

| ID | Quality | Requirement |
|---|---|---|
| NFR-01 | Correctness | All monetary values shall be integer rupees; every expense shall satisfy sum(splits) = total; group balances shall always sum to zero. |
| NFR-02 | Security | Passwords shall be hashed with bcrypt (cost 12); access tokens shall expire in 15 minutes; refresh tokens shall rotate and be stored only as hashes in an HttpOnly cookie. |
| NFR-03 | Security | Every group request shall verify active membership on the server; every request body shall be validated; authentication endpoints shall be rate limited. |

| ID | Quality | Requirement |
|---|---|---|
| NFR-04 | Performance | Receipt upload shall respond immediately (HTTP 202) and process in the background; savings matching and the store-price refresh shall never block the user interface. |
| NFR-05 | Reliability | OCR shall fall back to an offline engine; exchange rates and gold and silver prices shall fall back to the last saved copy; a store that cannot be read shall keep its previous prices; a failed notification or email shall never fail a money operation. |
| NFR-06 | Usability | Every list shall have loading, empty and error states; forms shall validate inline with human-readable messages; amounts shall display as "Rs 4,200". |
| NFR-07 | Portability | The application shall run in current desktop and mobile browsers and adapt to screens from 360 px wide. |
| NFR-08 | Maintainability | Business logic shall live in pure, testable service modules; the API shall use consistent response envelopes. |
| NFR-09 | Privacy | Receipt images shall be stored privately and served through signed URLs that expire after one hour. |
| NFR-10 | Deployability | Every change pushed to the main branch shall be tested automatically and deployed only if all tests and the client build pass. |
| NFR-11 | Politeness | The price refresh shall obey each store's robots.txt, identify itself with an honest User-Agent, and send no more than one request every two seconds to any store. |

## 3.4 Use Case Diagrams

Figures 3.1 and 3.2 present the use cases of the account and personal workspace and of the group workspace respectively.

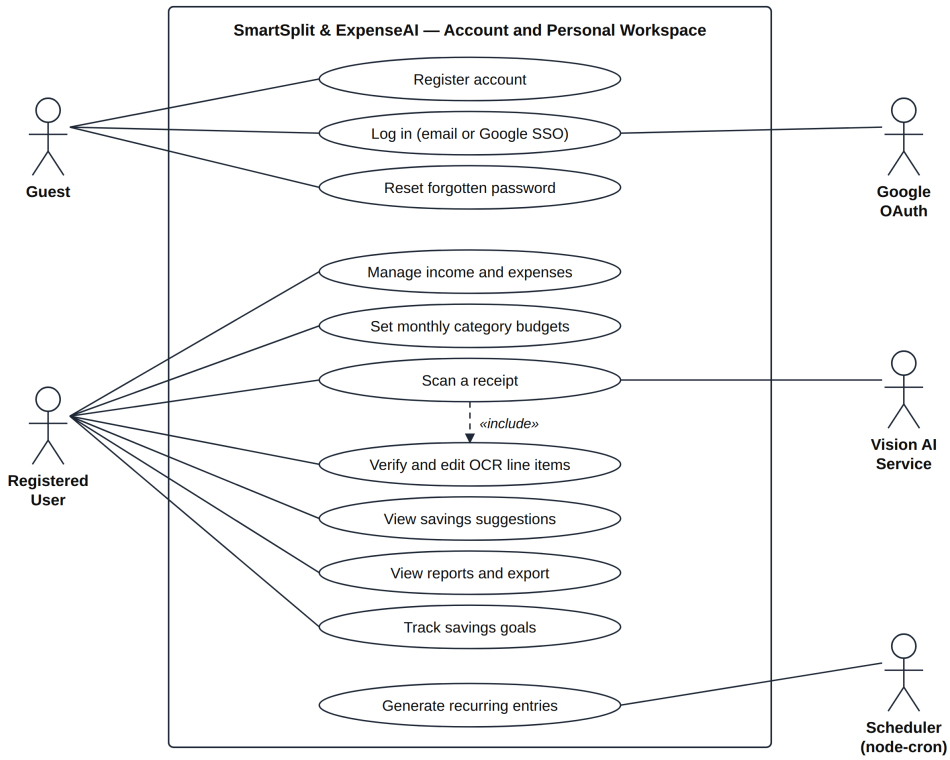



**Figure 3.1:** Use case diagram — account and personal workspace

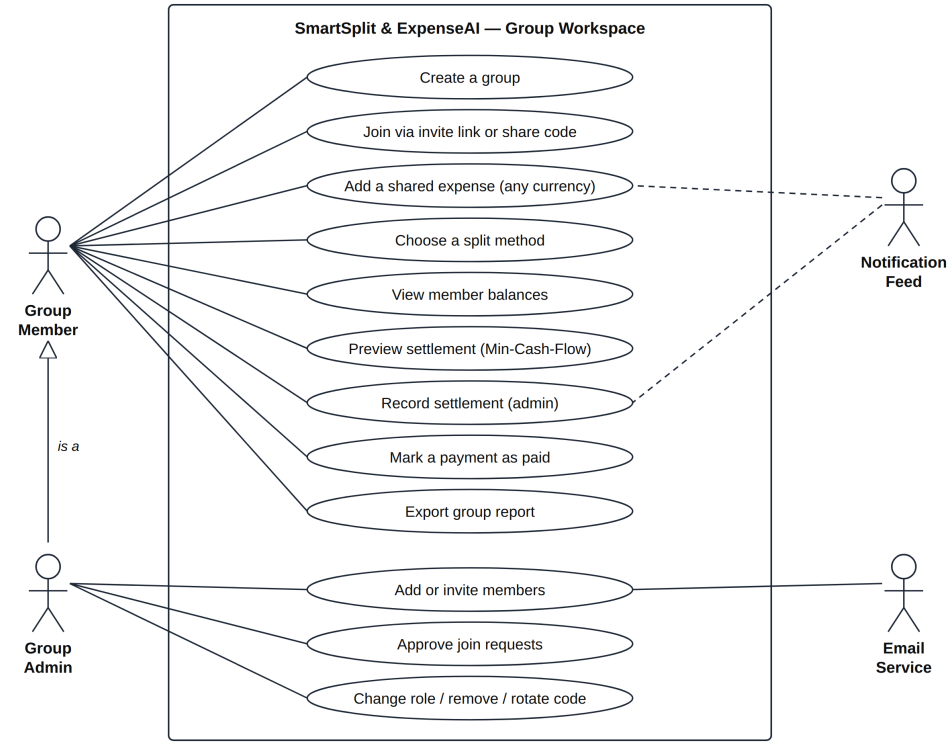



**Figure 3.2:** Use case diagram — group workspace

## 3.5 Use Case Descriptions

The three most important use cases are described below in tabular form.

**Table 3.4:** Use case UC-01: Add a shared expense

| | |
|---|---|
| **Use case ID** | UC-01 |
| **Name** | Add a shared expense |
| **Actor** | Group member |
| **Precondition** | The user is signed in and is an active member of the group. |
| **Main flow** | 1. The user opens Add Expense and selects the group.<br>2. The user enters title, amount, date and category.<br>3. The user selects one or more payers and their amounts.<br>4. The user selects a split method and fills in the shares.<br>5. The system validates the input live and enables Save.<br>6. The server validates the input again, computes the splits, stores the expense and notifies other members. |
| **Alternate flow** | 5a. Payers do not cover the total, exact amounts do not add up, or percentages do not total 100%: the system shows a message such as "You're Rs 50 short" and keeps Save disabled. |
| **Postcondition** | The expense is stored with sum(splits) = total, and the group balances reflect it. |

**Table 3.5:** Use case UC-02: Settle up a group

| | |
|---|---|
| **Use case ID** | UC-02 |
| **Name** | Settle up a group |
| **Actors** | Group member (preview), group admin (record) |
| **Precondition** | The group has at least one non-zero balance and no unpaid settlement in progress. |
| **Main flow** | 1. The user opens Settle Up.<br>2. The system shows the naive pairwise debts.<br>3. The user selects "Simplify with Min-Cash-Flow".<br>4. The system computes and displays the optimised payments and the number of transfers saved, without saving anything.<br>5. A group admin selects "Record settlement".<br>6. The system stores one settlement row per payment, marks the group as settled and writes an audit log entry. |
| **Alternate flow** | 4a. Nobody owes anything: the system reports "nothing to settle".<br>6a. A settlement is already in progress: the system refuses with a conflict message. |
| **Postcondition** | Payments are recorded; when each is marked paid, the corresponding balances return to zero. |

**Table 3.6:** Use case UC-03: Scan and verify a receipt

| | |
|---|---|
| **Use case ID** | UC-03 |
| **Name** | Scan and verify a receipt |

| | |
|---|---|
| **Actors** | Registered user, Vision AI service |
| **Precondition** | The user is signed in and has a photo of a printed receipt. |
| **Main flow** | 1. The user uploads or captures a photo.<br>2. The system stores the image and replies immediately.<br>3. The vision AI extracts store, date, items and total in the background.<br>4. The system shows the verification dialog with the image and editable lines.<br>5. The user corrects, adds or removes lines and confirms.<br>6. The system saves a personal transaction (or prepares a line-item group expense) and later attaches savings suggestions. |
| **Alternate flow** | 3a. The AI is unavailable: Tesseract.js extracts the lines offline.<br>4a. Lines and printed total differ by more than 5%: a warning shows both values. |
| **Postcondition** | Only user-verified data is stored. |

## 3.6 Business Rules

**Table 3.7:** Business rules enforced by the system

| ID | Rule |
|---|---|
| BR-01 | Money is stored and calculated as integer rupees; fractional rupee amounts are rejected. |
| BR-02 | Every expense satisfies sum(splits.share) = totalAmount and sum(paidBy.amount) = totalAmount. |
| BR-03 | Rounding remainders are assigned to the payer (the largest payer when there are several). |
| BR-04 | Balances are always computed from expenses and paid settlements; they are never stored as running totals. |
| BR-05 | Settlement uses greedy max-matching on net balances and produces at most V − 1 payments that bring all balances to zero. |
| BR-06 | Every group route verifies, on the server, that the requesting user is an active member; client-sent identifiers are never trusted. |
| BR-07 | OCR results are never saved automatically; the user verifies every line first. |
| BR-08 | Price suggestions read only the local benchmark collection and never block the user interface; live store prices are written into that collection by a separate background job. |
| BR-09 | A foreign-currency expense is converted to whole rupees on the server; the rounding difference goes to the largest payer or row so that BR-02 still holds, and an edit keeps the original rate. |
| BR-10 | Savings-goal progress is always computed from the recorded deposits and withdrawals, and a goal can never hold less than zero. |
| BR-11 | Recording a settlement closes the group, so only a group admin may do it. |
| BR-12 | A group's trip currency only sets the starting currency of new expenses; stored amounts, balances and settlements are always in rupees. |
| BR-13 | Zakat is 2.5% of net wealth (assets minus debts due now), rounded up to the next whole rupee, and is due only when net wealth is at or above the nisab. |
| BR-14 | A store price is accepted only for the same product: the same pack size within 2%, every word of the product name in the listing, and a price between 0.6 and 1.6 times the median already in the price book. |

## 3.7 Hardware and Software Requirements

**Table 3.8:** Hardware and software requirements

| Item | Development | End user |
|---|---|---|
| Processor / memory | Dual-core CPU, 8 GB RAM | Any device able to run a modern browser |
| Operating system | Linux, Windows or macOS | Any (Android, iOS, Windows, macOS, Linux) |
| Runtime | Node.js 20 or later, npm | Not required |
| Database | MongoDB 7 (local Docker) or MongoDB Atlas | Not required |
| Browser | Google Chrome (development and testing) | Chrome, Edge, Firefox or Safari (current versions) |
| Network | Internet for AI, image storage and Google sign-in | Internet connection |
| Tools | Visual Studio Code, Git, GitHub, Docker | None |

# CHAPTER 4
# SYSTEM DESIGN

This chapter describes the architecture of the system, its database design, the algorithms at its core, the receipt processing pipeline, the security design and the API conventions.

## 4.1 System Architecture

The system follows a three-tier client–server architecture (Figure 4.1). The **presentation tier** is a React single-page application that runs in the browser. The **application tier** is a stateless REST API [11] built with Node.js and Express. The **data tier** consists of MongoDB and a set of external services. The browser never communicates with the database or the AI service directly: every request passes through the server's security checks.

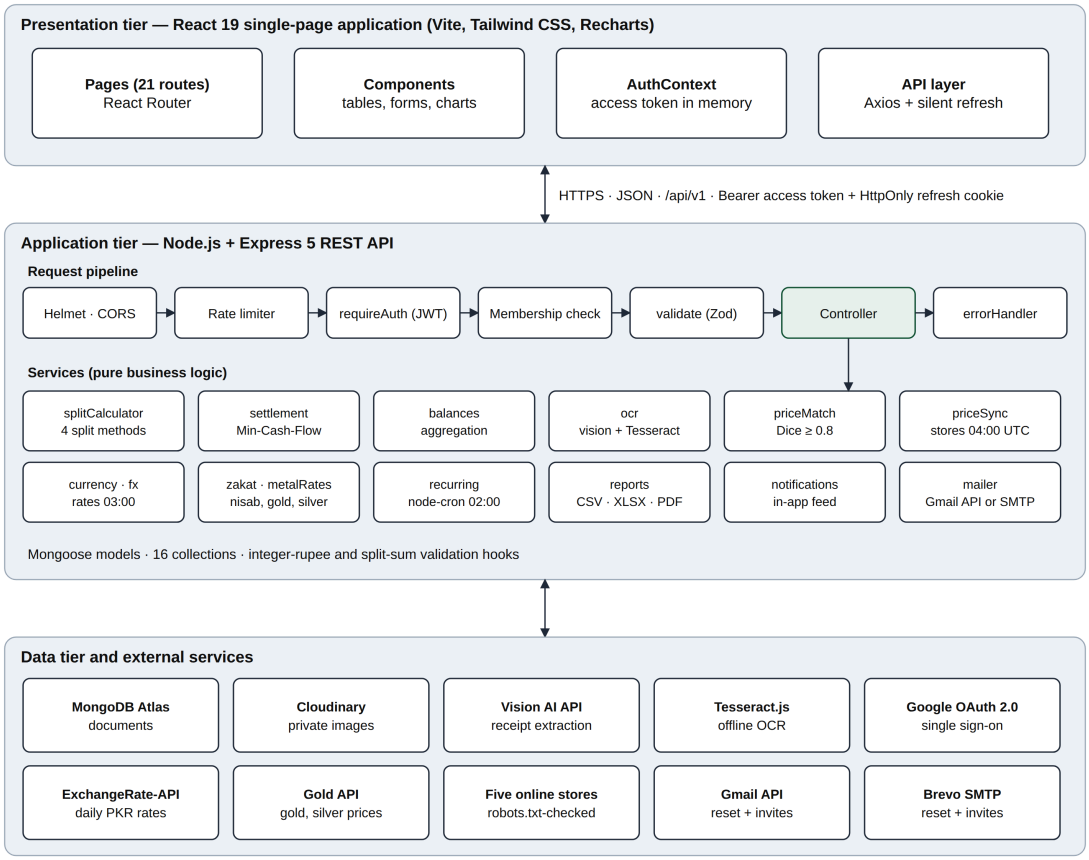



**Figure 4.1:** Three-tier architecture of SmartSplit & ExpenseAI

Each request passes through a fixed pipeline of middleware before it reaches a controller: security headers and CORS (Helmet), rate limiting, JWT authentication, group membership or admin checks, and Zod schema validation. Controllers are thin; the business logic lives in pure service modules (split calculation, settlement, balances, OCR, price matching, the store-price refresh, currency conversion and exchange rates, zakat and metal prices,

recurring entries, reports, notifications and email) that can be tested without a web server. Errors from any layer are converted by a single error handler into a consistent response envelope.

## 4.2 Database Design

MongoDB, a document database, was chosen because the central entity, a group expense, naturally contains nested lists: several payers, several split shares and several line items. Storing these inside one document means an expense is read and written atomically in a single operation. Mongoose [21] adds schemas, validation and indexes on top of MongoDB. Figure 4.2 shows the entity relationship diagram of the sixteen collections.

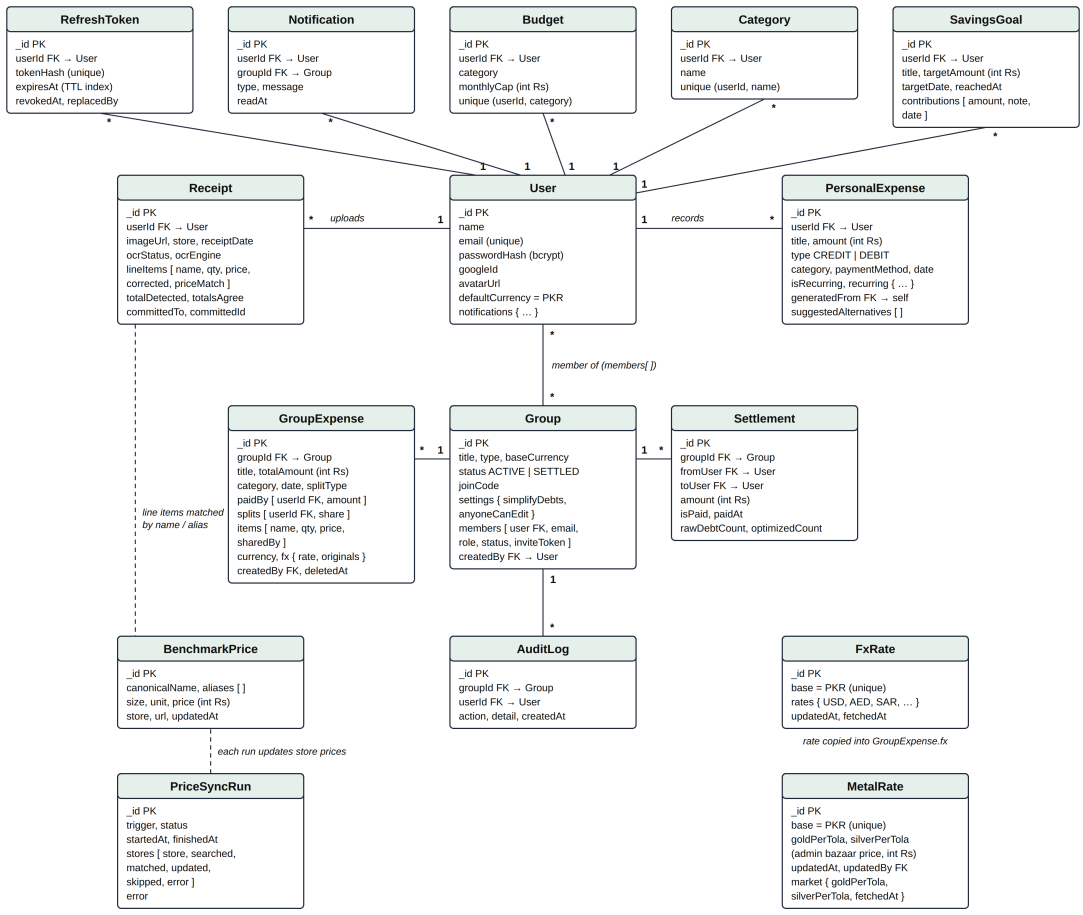



**Figure 4.2:** Entity relationship diagram

**Table 4.1:** Collections and their purpose

| Collection | Purpose and key constraints |
|---|---|
| User | Account details; unique email; bcrypt password hash (never returned to the client); optional Google ID. |
| RefreshToken | One row per session; stores only the SHA-256 hash of the token; TTL index removes expired rows automatically. |

| Collection | Purpose and key constraints |
|---|---|
| Group | Group details and an embedded members array (role admin/member; status active/pending/requested); share code. |
| GroupExpense | Expense with paidBy, splits and items arrays; a validation hook rejects any document whose shares do not sum to the total; optional fx sub-document with the original currency, amounts and rate; soft delete via deletedAt. |
| Settlement | One planned payment (from, to, amount, paid flag) plus the naive and optimised payment counts. |
| AuditLog | Append-only record of important group actions such as settlement creation. |
| PersonalExpense | Personal credit or debit; recurring template settings; generatedFrom links generated copies to their template. |
| Budget | Monthly cap per category per user; unique on (userId, category). |
| Category | User-defined categories; unique per user, case-insensitive. |
| Receipt | Image reference, OCR status and engine, extracted line items with correction flags and price matches. |
| BenchmarkPrice | Product name, aliases, pack size and unit, price, store and link; unique on (product, store). The bundled catalogue holds 100 products and 609 indicative price rows across seven stores, and the daily refresh writes a row for every product it finds at each of five online stores, overwriting the indicative row where the store is already in the catalogue. |
| Notification | One row per recipient with read state. |
| SavingsGoal | Goal title, target amount and date, and an embedded contributions array (positive deposits, negative withdrawals); reachedAt is set once. |
| FxRate | The latest rupee rates for the supported currencies with their date; a single row used as a fallback when the rate provider is unreachable. |
| MetalRate | A single row holding the last international gold and silver prices in rupees per tola with the time they were fetched (the fallback when the provider is unreachable), and the bazaar prices last saved by a super admin with who saved them and when. |
| PriceSyncRun | One row per store-price refresh: trigger (schedule, start-up, manual or command line), status, start and finish times, and for each store the number of requests made, products matched and prices updated, or the reason it was skipped or stopped. |

Indexes support the most frequent queries: expenses by group and date, groups by member, transactions by user and date, due recurring templates by next run date, unread notifications by user, and price-refresh runs by start time.

## 4.3 Expense Split Calculation

All split methods share one rounding rule so that every expense balances exactly. For a total T shared by n people, each person first receives the floor of T / n, and the remainder r is assigned to the payer:

$$share_i = \lfloor T / n \rfloor,\ r = T - n \cdot \lfloor T / n \rfloor,\ share_payer = \lfloor T / n \rfloor + r$$

For example, Rs 100 split among three people gives 34, 33 and 33, with the extra rupee assigned to the payer. Percentage splits use the floor of T × p / 100 for each person and assign the remainder in the same way. Exact splits must sum to the total, and line-item splits divide each item among the people who shared it and require the items to sum to the total. The calculation module is pure and is checked twice more: by the controller before saving and by a validation hook in the database model.

## 4.4 Settlement Algorithm (Min-Cash-Flow)

### *4.4.1 Net balance*

For each member u of a group, the net balance is computed from all non-deleted expenses and all settlement payments already marked as paid:

$$net(u) = \Sigma\ paid(u) - \Sigma\ owed(u) + \Sigma\ settled_out(u) - \Sigma\ settled_in(u)$$

A positive net means the group owes the member (a creditor); a negative net means the member owes the group (a debtor). Because every rupee paid is owed by someone, the nets always sum to zero; the algorithm verifies this and refuses to run otherwise, so a defect elsewhere can never create or lose money.

### *4.4.2 Greedy matching*

Figure 4.3 shows the algorithm as a flowchart, and the pseudocode is given below.

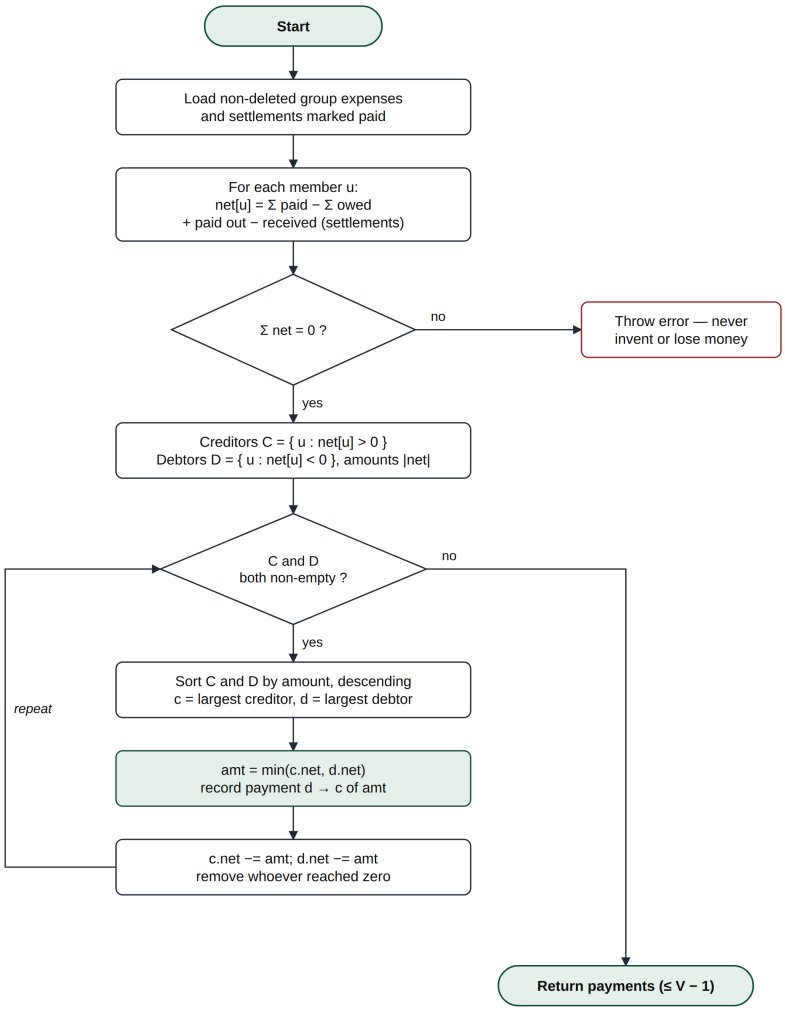



**Figure 4.3:** Flowchart of the Min-Cash-Flow settlement algorithm

```
MIN-CASH-FLOW(net)
  C ← { (u, net[u]) : net[u] > 0 }          // creditors
  D ← { (u, −net[u]) : net[u] < 0 }         // debtors
  if Σ C ≠ Σ D then error "balances do not net to zero"
  payments ← [ ]
  while C ≠ ∅ and D ≠ ∅ do
      sort C and D by amount, descending
      c ← C[0];  d ← D[0]
      amt ← min(c.amount, d.amount)
      append (d.user → c.user, amt) to payments
      c.amount ← c.amount − amt;  d.amount ← d.amount − amt
      remove c from C if c.amount = 0
      remove d from D if d.amount = 0
  return payments
```

### *4.4.3 Correctness and bound*

**Termination and bound.** In every iteration, amt equals the smaller of the two amounts, so at least one of the two members reaches zero and leaves the pool. After k payments at least k members have left. Since the balances sum to zero, when only one member remains that

member's balance must also be zero. Therefore the algorithm stops after at most V − 1 payments for V members with non-zero balances [1].

**Correctness.** Each payment reduces a debtor's debt and a creditor's credit by the same amount, so the sum of balances stays zero and every amount stays an integer. When the loop ends, both lists are empty and every balance is exactly zero.

**Complexity.** There are at most V − 1 iterations and each sorts at most V entries, giving $O(V^2 \log V)$ time and O(V) space. For groups of up to a few dozen members this is instantaneous; a binary heap would reduce it to $O(V \log V)$ for very large groups.

**Optimality.** The greedy result is not always the absolute minimum: when a subset of members already nets to zero among themselves, fewer payments may exist. Computing the true minimum is NP-hard [1], [2]; the V − 1 bound with polynomial running time is the accepted practical trade-off.

### *4.4.4 Worked example*

The demonstration group "Murree Trip" has five members and seven expenses totalling Rs 49,600. Table 4.2 shows the net balances computed by the system and Table 4.3 shows each iteration of the algorithm.

**Table 4.2:** Net balances of the Murree Trip group

| Member | Net balance (Rs) | Role |
|---|---|---|
| Sara Khan | +11,480 | Creditor |
| Bilal Ahmed | +7,580 | Creditor |
| Azeem Shahid | −3,520 | Debtor |
| Hamza Tariq | −7,420 | Debtor |
| Ali Raza | −8,120 | Debtor |
| **Total** | **0** | |

**Table 4.3:** Iterations of Min-Cash-Flow on the Murree Trip group

| Step | Largest creditor | Largest debtor | Payment and remainder |
|---|---|---|---|
| 1 | Sara +11,480 | Ali −8,120 | Ali → Sara Rs 8,120; Sara +3,360, Ali settled |
| 2 | Bilal +7,580 | Hamza −7,420 | Hamza → Bilal Rs 7,420; Bilal +160, Hamza settled |
| 3 | Sara +3,360 | Azeem −3,520 | Azeem → Sara Rs 3,360; Sara settled, Azeem −160 |
| 4 | Bilal +160 | Azeem −160 | Azeem → Bilal Rs 160; both settled |

The group is settled in four payments, which equals the bound V − 1 = 4. Repaying each share of each bill individually would have required 20 separate transfers, so the optimiser saves 16 transfers.

## 4.5 Receipt Processing Pipeline

Figure 4.4 shows the sequence of interactions when a receipt is scanned. The upload request returns HTTP 202 (Accepted) immediately, and extraction runs in the background while the client polls for the result. The vision model is called with **structured output**: the response must match a fixed schema (store, date, items with name, quantity and line price, and total), so it cannot return free text or malformed JSON. The prompt fixes the domain rules: whole rupees, line totals rather than unit prices, subtotal and tax rows skipped, and null rather than a guess for unreadable values. If the call fails for any reason, Tesseract.js produces the same structure from raw text using line-pattern rules, and the receipt records which engine was used.

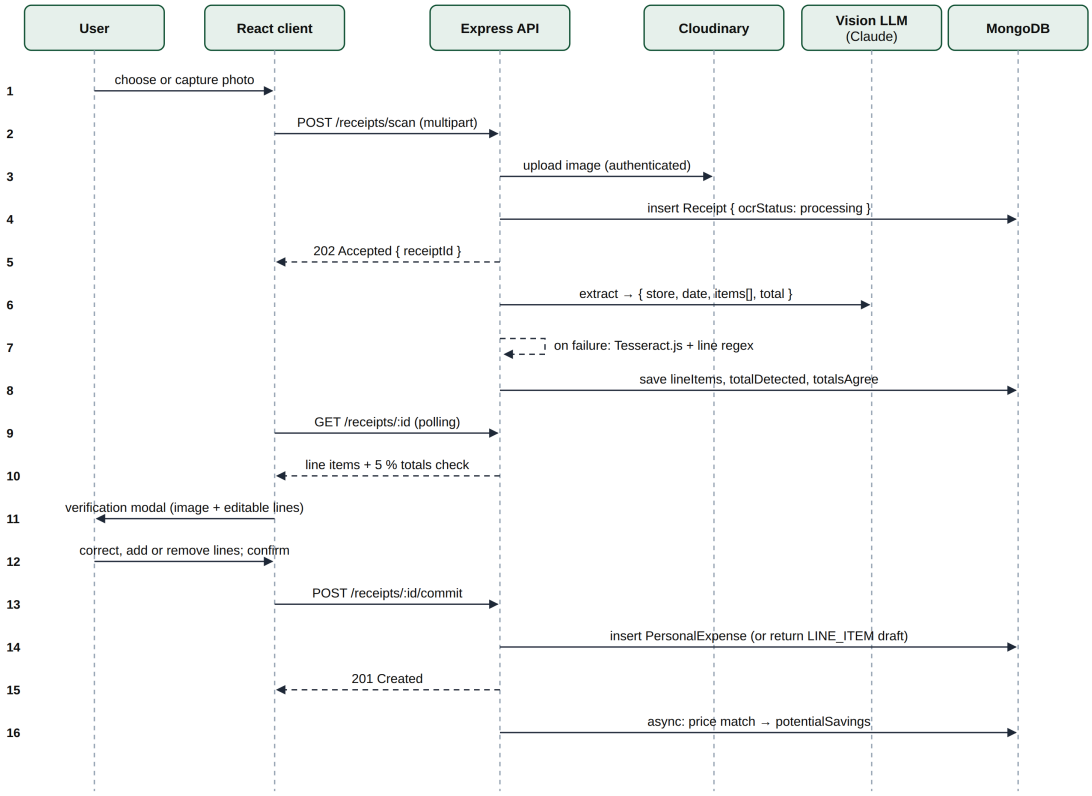



**Figure 4.4:** Sequence diagram of receipt scanning and verification

After extraction, the sum of the line prices is compared with the printed total; a difference above 5% produces a warning that names both values. Nothing is written as a transaction until the user confirms the lines in the verification dialog. Committing to a group does not create the expense; it returns a line-item draft so the user still chooses who shared each item.

## 4.6 Price Matching and Savings Suggestions

Each verified line item is matched against the benchmark collection in four steps:

1. **Normalisation** — the name is lower-cased and pack sizes, units, punctuation and filler words are removed, so "COOKING OIL 3LTR @" and "Cooking Oil 3 Litre" produce the same key.
2. **Alias lookup** — an indexed exact match against known receipt spellings, for example "anday" → Eggs (dozen) or "surf excel" → Detergent 1 kg.
3. **Fuzzy match** — otherwise the Dice coefficient [5] is computed, and a match is accepted only at 0.8 or above.
4. **Size check** — pack sizes are parsed separately (kg ↔ g, L ↔ ml) and must agree within 2%, so a 5 kg bag is never compared with a 1 kg bag.

$$Dice(a, b) = 2 \cdot |bigrams(a) \cap bigrams(b)| \,/\, ( |bigrams(a)| + |bigrams(b)| )$$

Among matching products, the cheapest platform is selected, comparing only prices updated in the last seven days when the product has any and every recorded price otherwise, so an out-of-date figure cannot undercut a store's current price; if the user paid more, the saving (multiplied by quantity) is recorded and up to three cheaper alternatives are attached to the transaction. Matching runs after the save has been acknowledged, so it never delays the user. The benchmark collection itself is refreshed every morning from five online stores by a separate background job (Section 5.3.5), so matching always reads local data and never waits for a store.

## 4.7 Multi-Currency Conversion

Trips abroad produce bills in other currencies. The user may enter a group expense in one of 15 foreign currencies (for example AED, SAR, USD or GBP) with up to two decimal places. The server converts every amount to rupees at the current rate R (rupees per unit of the foreign currency) and stores the expense in whole rupees, so balances and settlement are unchanged. Rates come from a free, keyless exchange-rate service [22], refreshed every day at 03:00 and stored in the database as a fallback. Each converted value is rounded separately, so their sum may differ from the converted total by a rupee or two; that difference is given to the largest payer (or the largest row of an exact or line-item split):

$$T_PKR = round(T \times R),\ p_i = round(a_i \times R),\ p_largest \mathrel{+}= T_PKR - \Sigma\, p_i$$

Table 4.4 shows the conversion of a desert safari on the demonstration Dubai trip at the rate on 30 October 2026. The original amounts and the rate are kept in an `fx` sub-document, so the expense list can show "AED 412.5" under the rupee amount, and editing the expense later keeps the rate it was added at.

**Table 4.4:** Conversion of AED 412.50 shared equally by four members at 1 AED = Rs 75.2955

| Value | In AED | In rupees | Calculation |
|---|---|---|---|
| Total, paid by Azeem | 412.50 | 31,059 | 412.50 × 75.2955 = 31,059.39, rounded |
| Share of Sara, Bilal and Ali | 103.125 each | 7,764 each | ⌊31,059 / 4⌋ |
| Share of Azeem (payer) | 103.125 | 7,767 | 7,764 + remainder 3 (BR-03) |
| Sum of shares | 412.50 | 31,059 | Equals the stored total (BR-02) |

## 4.8 Savings Goal Progress

A savings goal stores only its target and the list of deposits and withdrawals; everything the user sees is computed when the goal is read, in the same spirit as group balances. For a goal of target A due on date D, created on date C, with saved amount S on date t:

$$percent = \lfloor 100 \cdot S / A \rfloor, \; monthly = \lceil (A - S) / \lceil days_left / 30 \rceil \rceil$$

$$on\ track \;\Leftrightarrow\; S \geq A \cdot (t - C) / (D - C)$$

The "on track" test compares the saved amount with a straight line from nothing on the day the goal was created to the full target on its date. When the saved amount first reaches the target, the goal records the date and the owner receives a single GOAL_REACHED notification; a withdrawal that would take the goal below zero is refused.

## 4.9 Security Design

Authentication uses two tokens with different lifetimes and storage (Table 4.5). Figure 4.5 shows sign-in, transparent token refresh and reuse detection.

**Table 4.5:** Token design

| Token | Lifetime | Storage | Purpose |
|---|---|---|---|
| Access token (JWT) | 15 minutes | Browser memory only (never localStorage) | Sent as a Bearer header with every API request |
| Refresh token | 7 days | HttpOnly, SameSite=Strict, Secure cookie scoped to /api/v1/auth; only its hash stored server-side | Obtains a new access token; rotated on every use |

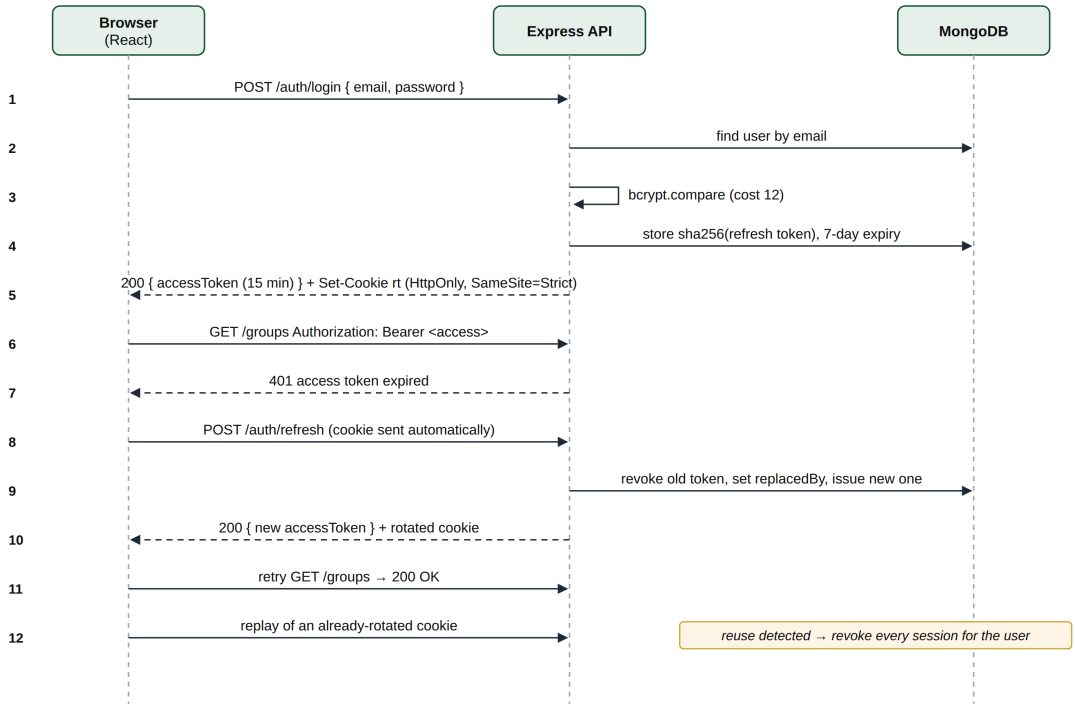



**Figure 4.5:** Sequence diagram of sign-in, token refresh and reuse detection

If a refresh token that has already been rotated is presented again, it can only be a copy, so all of that user's sessions are revoked. Further controls include bcrypt hashing at cost 12 [8]; identical responses for unknown emails and wrong passwords; single-use password reset tokens stored as SHA-256 hashes with a 30-minute expiry; Zod validation of every request body, parameter and query; per-route and global rate limits; Helmet security headers; CORS restricted to the client origin; private, signed receipt image URLs; and a start-up guard that refuses to run in production with missing, weak or placeholder secrets. Super administrators are defined by the `ADMIN_EMAILS` server setting rather than a database field, so the role cannot be granted through the API; it guards the Admin overview, changes to the price book, saving bazaar gold and silver prices, and starting a store-price refresh. These controls address the OWASP categories of broken access control, cryptographic failures, injection and authentication failures [9].

## 4.10 API Design

The API exposes 84 REST endpoints under the prefix `/api/v1`, grouped by resource (authentication, groups and invitations, expenses, settlements, personal transactions and budgets, savings goals, zakat, receipts, reports and notifications, categories, users, exchange rates, the price book and administration). Every response uses one of two envelopes:

```
{ "success": true,  "data": { ... } }
{ "success": false, "error": { "code": "SPLIT_INVALID", "message": "You're Rs 50
short" } }
```

Machine-readable error codes (for example `NOT_A_MEMBER`, `TOKEN_REUSED`, `SETTLEMENT_IN_PROGRESS`) let the client react precisely, while the human-readable message is shown to the user unchanged. The complete endpoint list is given in Appendix A.

## 4.11 User Interface Design

The interface uses a clean, card-based layout with a fixed top navigation (Dashboard, Groups, Personal, Goals, Scan and Reports, plus Admin for super administrators) and a notification bell; the Zakat calculator is opened from the Goals page. Colour carries meaning consistently: green for money owed to the user or within budget, amber for budgets above 80%, and red for debts and overspending. Every list implements loading, empty and error states; destructive actions ask for confirmation; and amounts are always formatted as "Rs 4,200". Layouts collapse to a single column on narrow screens, and wide tables scroll horizontally within their cards.

# CHAPTER 5
# IMPLEMENTATION

This chapter describes the tools, project structure and implementation of the main modules, and presents the user interface.

## 5.1 Technology Stack

**Table 5.1:** Technologies used

| Layer | Technology | Role in the system |
|---|---|---|
| Frontend | React 19, Vite | Component-based single-page application with fast development builds |
| Frontend | React Router | Client-side routing across 21 routes with protected routes |
| Frontend | Tailwind CSS | Utility-first, responsive styling |
| Frontend | Recharts, Lucide | Charts on the Reports page; icon set |
| Frontend | Axios | HTTP client with interceptors for tokens and silent refresh |
| Backend | Node.js, Express 5 | REST API server and middleware pipeline |
| Database | MongoDB, Mongoose | Document storage, aggregation pipelines, schemas and validation |
| Validation | Zod | Schema validation of every request |
| Security | jsonwebtoken, bcryptjs, Helmet, express-rate-limit | Tokens, password hashing, headers and rate limiting |
| Identity | Google Identity Services / google-auth-library | Google OAuth 2.0 sign-in |
| AI / OCR | Anthropic Claude vision (SDK), any OpenAI-compatible vision model, Tesseract.js | Structured receipt extraction, a free AI option, and an offline fallback |
| Files | Multer, Cloudinary | Upload handling and private image storage |
| Jobs | node-cron | Nightly recurring-transaction job (02:00), exchange-rate refresh (03:00) and store-price refresh (04:00 UTC, 09:00 in Pakistan) |
| Exchange rates | ExchangeRate-API (open access) | Daily rupee rates for 15 currencies, no API key |
| Metal prices | Gold API | International gold and silver prices in US dollars per troy ounce, no API key |
| Store prices | Al-Fatah, GrocerApp, Metro, Jalal Sons, Chase Up | Daily prices read from store search JSON, a public search API, and sitemaps with schema.org Product data, within each robots.txt |
| Exports | json2csv, ExcelJS, PDFKit | CSV, XLSX and PDF generation |

| Layer | Technology | Role in the system |
|---|---|---|
| Email | Nodemailer, Gmail API, Brevo SMTP | Password reset and invitation emails; the Gmail API over HTTPS is used first when configured |
| Testing | Vitest, Supertest, mongodb-memory-server | Unit, API and property-based tests on an in-memory database |
| Hosting | Render, MongoDB Atlas | One web service for the API and the built client; managed database |
| CI/CD | GitHub Actions | Runs every test and the client build on each push; Render deploys only if they pass |

## 5.2 Project Structure

The code is organised as a monorepo with separate client and server packages. The server contains approximately 6,600 lines of application code and 4,400 lines of tests; the client contains approximately 6,400 lines across 20 pages and 27 reusable components.

```
smartsplit/
├── client/                 React + Vite application
│   └── src/
│       ├── pages/          one component per route (20)
│       ├── components/     reusable UI (27)
│       ├── api/            Axios instance and API functions
│       └── context/        AuthContext, CategoriesContext
└── server/                 Express API
    └── src/
        ├── models/         Mongoose schemas (16)
        ├── routes/         routers + Zod schemas per resource
        ├── controllers/    request handlers
        ├── services/       settlement, splitCalculator, balances, ocr,
        │                   priceMatch, priceSync, currency, fx, zakat,
        │                   metalRates, recurring, reports, mailer, ...
        ├── middleware/     auth, membership, validate, rate limits, errors
        ├── scripts/        seed, prices:sync, gmail:token
        └── tests/          Vitest + Supertest suites (24 files)
```

## 5.3 Backend Implementation

### *5.3.1 Settlement service*

The settlement service implements the algorithm of Section 4.4 as a pure function. Its core loop is shown below.

```
while (creditors.length && debtors.length) {
  creditors.sort(byLargest);
  debtors.sort(byLargest);

  const creditor = creditors[0];
  const debtor = debtors[0];
  const amount = Math.min(creditor.net, debtor.net);

  payments.push({ from: debtor.userId, to: creditor.userId, amount });

  creditor.net -= amount;
  debtor.net -= amount;

  // At least one of the two is now square and leaves the pool.
  if (creditor.net === 0) creditors.shift();
  if (debtor.net === 0) debtors.shift();
}
```

The same net balances are also computed inside MongoDB with an aggregation pipeline ($unwind and $group over the paidBy and splits arrays, combined with a $facet), which is used by the live routes for efficiency [12]. An automated test runs both implementations on the same ledger and asserts identical results, preventing the two from drifting apart.

### *5.3.2 Split calculator*

The split calculator distributes an integer total with the floor-and-remainder rule:

```
function distribute(total, userIds, preferredUserId) {
  const headCount = userIds.length;
  const base = Math.floor(total / headCount);
  const shares = new Map(userIds.map((id) => [String(id), base]));

  const leftover = total - base * headCount;
  if (leftover > 0) {
    const target = preferredUserId && shares.has(String(preferredUserId))
      ? String(preferredUserId) : String(userIds[0]);
    shares.set(target, shares.get(target) + leftover);
  }
  return shares;
}
```

### *5.3.3 Authentication and authorisation*

The `requireAuth` middleware verifies the Bearer access token and loads the user. The `requireMember` middleware loads the group from the URL identifier and looks up the caller's own active membership using the authenticated user identifier only; a non-member receives HTTP 403 and an unknown group HTTP 404. `requireAdmin` additionally checks the admin role; it guards member management and, because recording a settlement closes the group, settlement recording as well. Refresh-token rotation with reuse detection is implemented as follows:

```
const existing = await RefreshToken.findOne({ tokenHash: hash(token) });
if (!existing) return { error: 'INVALID' };

// A rotated token presented again is evidence the cookie leaked.
if (existing.revokedAt && existing.replacedBy) {
  await RefreshToken.updateMany({ userId: existing.userId, revokedAt: null },
                                { revokedAt: new Date() });
  return { error: 'REUSED' };
}
if (existing.revokedAt || existing.expiresAt < new Date()) return { error:
'INVALID' };

const next = await issueRefreshToken(existing.userId);
existing.revokedAt = new Date();
existing.replacedBy = hash(next.token);
await existing.save();
```

### *5.3.4 Receipt OCR and price matching*

Uploads are received in memory by Multer, limited to 8 MB and image types, and stored on Cloudinary as authenticated assets. The OCR service tries up to three engines in order: the Claude vision model through the Anthropic SDK with a Zod-defined output format; otherwise any OpenAI-compatible vision model (for example a free model on OpenRouter or Google AI Studio), whose reply is parsed leniently and validated with the same schema; and finally Tesseract.js with its English language data bundled locally, so it needs no download and a failure can never stop the server. For the offline engine, a regular expression extracts lines such as "Cooking Oil 3L 1,745", "2 x Milk 220" and quantity–rate–amount columns. The price-matching service implements normalisation, alias lookup, the Dice coefficient (shown below) and size parsing.

```
export function similarity(leftText, rightText) {
  if (leftText === rightText) return 1;
  if (leftText.length < 2 || rightText.length < 2) return 0;
  const first = bigrams(leftText);
  const second = bigrams(rightText);
  let shared = 0;
  for (const [pair, count] of first) shared += Math.min(count, second.get(pair) ||
0);
  return (2 * shared) / (leftText.length - 1 + (rightText.length - 1));
}
```

### *5.3.5 Live store prices*

The project rule that live price collection must be a decoupled background job with the local collection as a cache is implemented by the price-refresh service. It runs every day at 04:00 UTC (09:00 in Pakistan); on start-up, unless a run that began in the previous 20 hours has finished or is still in progress; from the "Update now" button on the Admin page; and from a terminal with `npm run prices:sync`. For every product in the price book it looks for that product at five online stores and writes what it finds into the same `benchmark_prices` collection that the receipt matcher reads, so no code that reads prices had to change. Al-

Fatah, Metro and Chase Up use the same store names as the bundled catalogue, so their live prices overwrite its indicative figures; GrocerApp and Jalal Sons prices are added as new rows. Table 5.2 shows how each store is read.

**Table 5.2:** Online stores read by the price refresh

| Store | Source | Notes |
|---|---|---|
| Al-Fatah | The Shopify storefront's product search, which returns JSON (`/search/suggest.json`) | Sold-out products are ignored |
| GrocerApp | The public search API that its own website calls (`/v15/search`) | Gives names, pack units and whole-rupee prices without opening product pages |
| Metro | Sitemap, then the schema.org Product data on product pages (`/detail/`) | Its search API requires an obfuscated signed header, so only the sitemap is used |
| Jalal Sons | Sitemap, then the schema.org Product data on product pages (`/product/`) | Product pages carry structured price and stock data |
| Chase Up | Sitemap, then the schema.org Product data on product pages (`/product/`) | Same shop platform as Jalal Sons |

Before a store is used, its `robots.txt` [23] is fetched and every path the adapter needs is checked against the rules for all user agents, the longest matching rule deciding; if any path is disallowed, the store is skipped for that run and the reason is recorded (following RFC 9309, a store without a robots.txt is treated as having no rules, while one whose robots.txt cannot be reached is not read at all that day). For this reason Daraz, whose robots.txt disallows `/catalog/` (its search pages), and Naheed, which disallows `/catalogsearch/`, are not read at all. Every request carries an honest User-Agent (`SmartSplitFYP/1.0 (student project; +https://smarsplit.onrender.com)`) and a 20-second time-out, and each store receives one request at a time with a two-second pause before each; the five stores are on different hosts and are read side by side. After three consecutive failed requests, or if its sitemap cannot be read, a store is left until the next run and keeps its previous prices.

Search stores are queried with the pack size first ("Cooking Oil 3 ltr") and then without it. For sitemap stores the product name is taken from each page address, and only the pages whose names match a book product are opened, at most four per product, shortest names first; the price and availability then come from the page's schema.org Product data [24]. A listing is accepted only if it has the same pack size as the book product (within 2%), contains every word of the product name, and is priced between 0.6 and 1.6 times the median price already in the book, which stops "oil 3 L" from matching engine oil. Of the listings that fit, the middle-priced one becomes the store's price, because during development the cheapest was often a different grade, such as broken basmati rice, and would have shown a saving that does not exist:

```
const fits = listings.filter((listing) =>
  listing.price > 0 && sameProduct(product, listing.title) &&
  (!median || (listing.price >= median * 0.6 && listing.price <= median * 1.6)));
fits.sort((left, right) => left.price - right.price);
return fits[Math.floor((fits.length - 1) / 2)] ?? null;   // the middle price
```

Each run is recorded in the `price_sync_runs` collection with, for every store, the number of requests made, products matched and prices updated, or why the store was skipped or stopped; the Admin page shows the latest run (Figure 5.16). Only one run can be in progress at a time: a second request returns the run that is already under way. No user request ever waits for a store, because receipts are always matched against the local collection.

Two refreshes were run against the live stores on 2 November 2026, and each took about seven minutes. The first updated 191 prices (Al-Fatah 41, GrocerApp 38, Metro 29, Jalal Sons 45 and Chase Up 38 of the 100 products) without a single failed store. In the second, shown in Figure 5.16, Chase Up answered HTTP 500 to three requests in a row after 25 of its prices had been updated, so the refresh stopped reading it and kept its remaining prices, while the other four stores completed normally.

### *5.3.6 Multi-currency expenses*

The currency service is a pure function that receives a validated expense in its entry currency and the rate, and returns the same expense in rupees together with the original figures. Payers, exact shares and line items are first checked in cents of the entry currency, so a message such as "Payers are AED 40 short" is given in the currency the user typed. The helper below rounds each value and moves the rounding difference to one chosen row:

```
function toRupees(values, rate, totalRupees, target) {
  const rupees = values.map((value) => Math.round(value * rate));
  rupees[target] += totalRupees - rupees.reduce((sum, value) => sum + value, 0);
  return rupees;
}
```

The exchange-rate service keeps the latest rates in memory for 12 hours and in the `fx_rates` collection. If the provider cannot be reached, the saved copy is used and marked as stale, and a failed attempt is not repeated for ten minutes; only when no rates were ever saved does the API answer HTTP 503 with the code `FX_UNAVAILABLE`.

### *5.3.7 Trip currency*

When a group is created, its trip currency is chosen from a drop-down list of the 16 currencies that the exchange-rate service can convert (`CURRENCIES` in `fx.js`), and the Zod schemas for creating and updating a group accept nothing else:

```
baseCurrency: z.enum(CURRENCIES, { message: 'Pick a currency from the list'
}).default('PKR'),
```

When Add Expense opens for a new expense in that group, the entry currency starts as the trip currency, so on a Dubai trip the amount is typed in AED straight away; an expense being edited keeps its own currency, and one built from a scanned receipt stays in rupees. The trip currency never changes how money is stored: every amount, balance and settlement figure is still in rupees and labelled "Rs", with the original foreign amount shown beneath a converted expense.

### *5.3.8 Savings goals*

The goals controller exposes six endpoints (list, create, edit, delete, add a contribution and remove a contribution). Every query includes the owner's identifier, so a goal belonging to someone else answers HTTP 404. Progress is computed by a pure `progressOf(goal, now)` function that implements the formulas of Section 4.8, and is unit-tested with fixed dates.

### *5.3.9 Zakat calculator*

The Zakat page estimates the zakat due on the user's wealth. Its form starts with figures the application already holds: money in savings goals, what group members owe the user, and what the user owes in groups; the difference between the user's income and spending over the last twelve months is shown only as a hint. The user adds cash, bank balances, gold and silver in tola, business stock, investments and other debts due now, and chooses the nisab: 52.5 tola of silver (the default, which gives the lower threshold) or 7.5 tola of gold. The pure `calculateZakat` function then works in whole rupees:

$$W = max(assets - debts, 0),\ nisab = round(N \times P),\ zakat = \lceil W / 40 \rceil \text{ if } W \geq nisab, \text{ otherwise } 0$$

Here N is the nisab in tola and P the price per tola of the chosen metal. Dividing by 40 gives 2.5%, and rounding up means the amount is never a rupee short: with silver at Rs 3,000 per tola the nisab is 52.5 × 3,000 = Rs 157,500, so wealth of Rs 157,499 owes nothing, while exactly Rs 157,500 owes ⌈3,937.5⌉ = Rs 3,938.

Gold and silver prices come from Gold API [25], a free service without a key that quotes US dollars per troy ounce. They are converted to rupees per tola (11.664 g) with the current dollar rate from the exchange-rate service:

$$P_tola = round(USD_per_ounce / 31.1035 \times 11.664 \times R_USD)$$

As with exchange rates, the result is kept in memory for 12 hours and saved in the `metal_rates` collection; when the provider cannot be reached the saved copy is used and marked as stale, and a failed attempt is not repeated for ten minutes. Bazaar (sarafa) prices in Pakistan are usually a little above the international price, so a super admin can save bazaar prices for everyone; they are used for as long as they are newer than the latest international price. Any user can also type their own prices for a single calculation, and if a price that the calculation needs is missing the API answers HTTP 400 with the code

ZAKAT_RATE_MISSING. Nothing is saved automatically: "Record as paid" asks for confirmation and then adds a "Zakat <year>" money-out transaction (category Misc, paid in cash) to the Personal workspace, where it can be edited.

#### *5.3.10 Recurring transactions, reports and notifications*

A node-cron job runs every night at 02:00 and calls `runDueRecurring(now)`, which finds every template whose next run date has passed, writes one entry per missed period (so a server that was offline for three months catches up with three entries) and advances the schedule; a 120-iteration guard prevents runaway loops. Reports are computed with aggregation pipelines and exported with json2csv, ExcelJS and PDFKit. Notifications are written as one row per recipient on four events; the notification function catches its own errors so that a failure can never cancel the money operation that triggered it.

#### *5.3.11 Email through the Gmail API*

Password-reset and invitation emails were first sent through Brevo SMTP. As of 2 November 2026 Brevo had still not activated the account for sending, and Render's free tier blocks the outgoing SMTP ports, so the mailer can also send through the Gmail API [26] over HTTPS. It uses the first transport that is configured: the Gmail API when the three `GMAIL_*` settings are present, otherwise SMTP, otherwise (in development) it prints the message to the server log. For Gmail, the stored refresh token is exchanged for an access token, which is cached until a minute before it expires; Nodemailer's stream transport builds the message, and the raw message is posted, base64url-encoded, to the Gmail send endpoint. The refresh token is obtained once with `npm run gmail:token`, a local OAuth sign-in limited to the `gmail.send` scope, so the server can send mail but cannot read the mailbox. If Gmail refuses a message, the reason is logged and the error is passed to the caller, which treats the email as not delivered: the invitation link is then offered for copying or sharing on WhatsApp, and a password-reset request still receives the usual answer.

## 5.4 Frontend Implementation

The client is built from function components and hooks. An `AuthContext` holds the signed-in user and keeps the access token in memory; on start-up it attempts a refresh so a page reload keeps the session. A shared Axios instance attaches the token, unwraps the response envelope and, on HTTP 401, performs a single refresh shared by all concurrent requests before retrying them. Pages never call Axios directly; they use typed functions in the `api` folder. A `useAsync` hook and a shared `States` component provide loading, empty and error states for every list, and the Reports page is lazy-loaded to keep the initial bundle small. An error boundary around each page reloads the application once if a newer deployment has replaced the page's code, instead of showing a blank screen.

## 5.5 User Interface

Figures 5.1 to 5.18 show the main screens of the application running with the demonstration data.

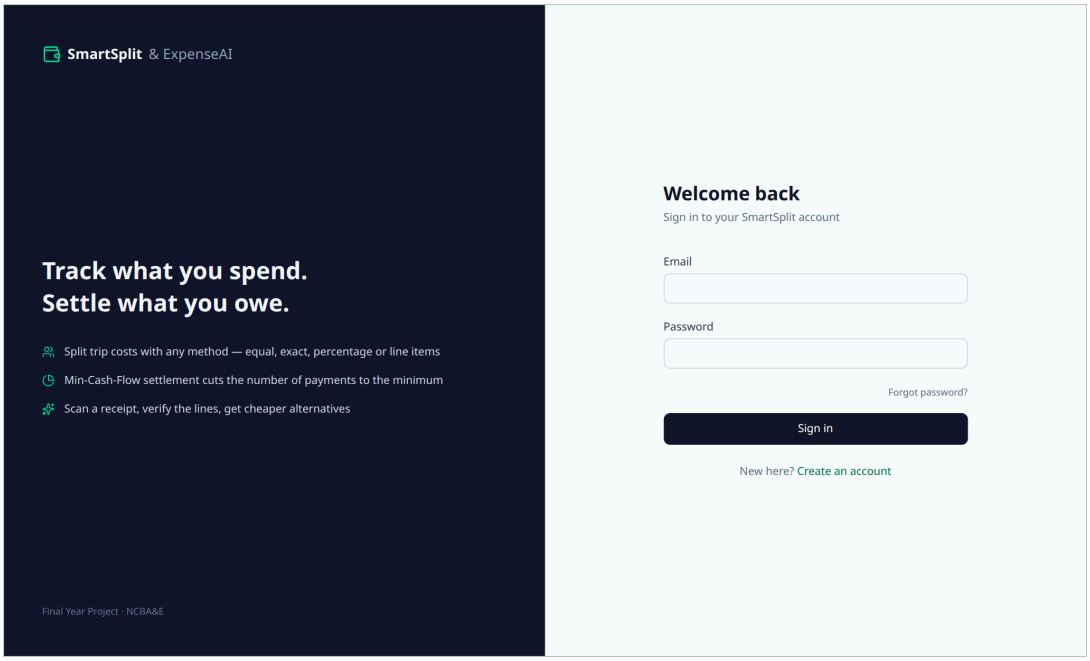



**Figure 5.1:** Sign-in page

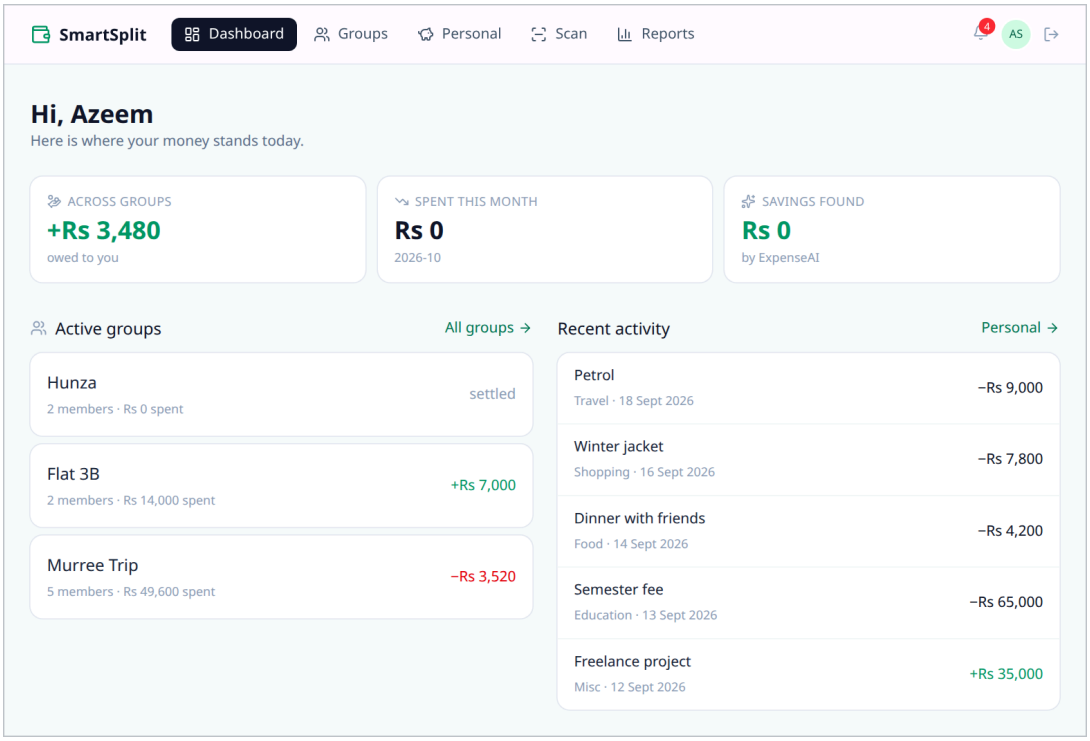



**Figure 5.2:** Dashboard combining both workspaces

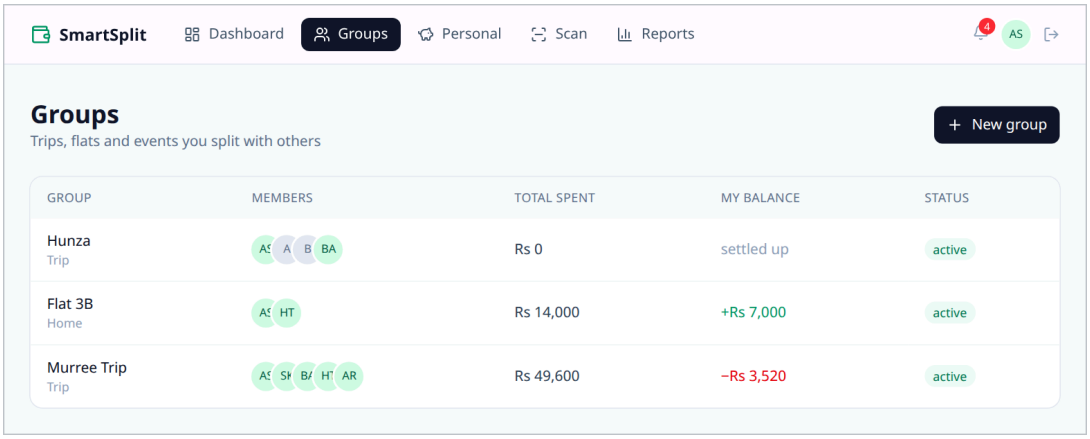



**Figure 5.3:** Groups list with total spent and personal balance

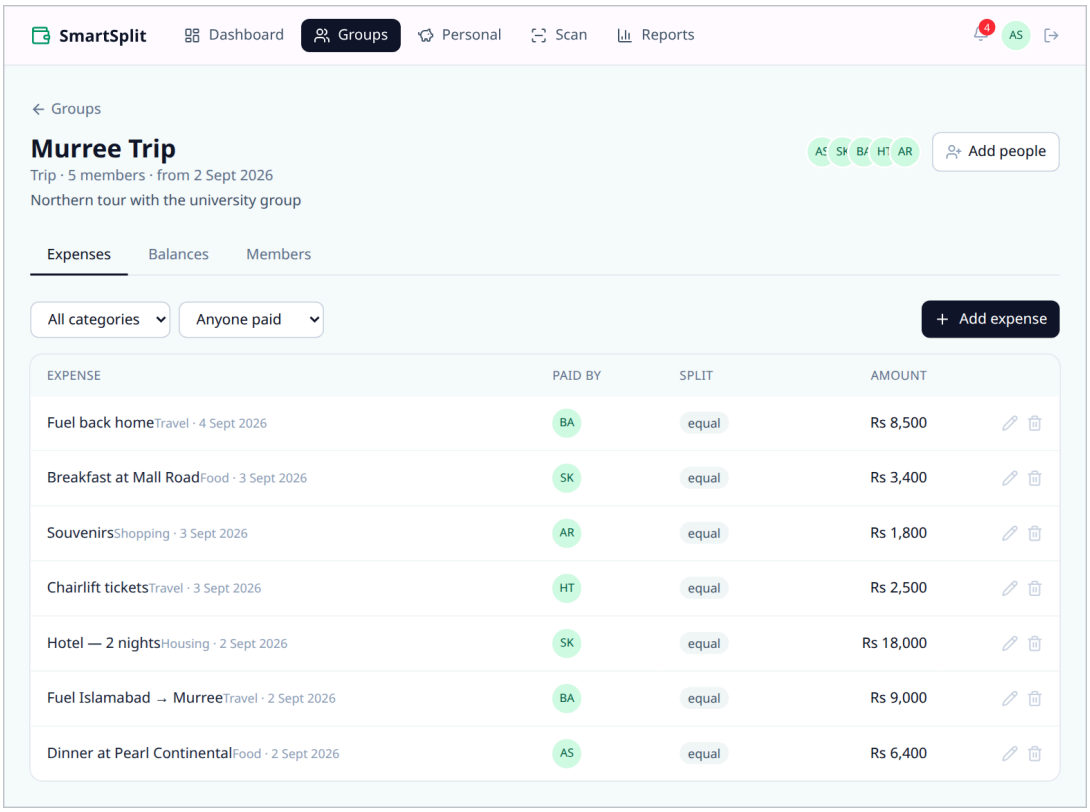



**Figure 5.4:** Group expenses of the Murree Trip

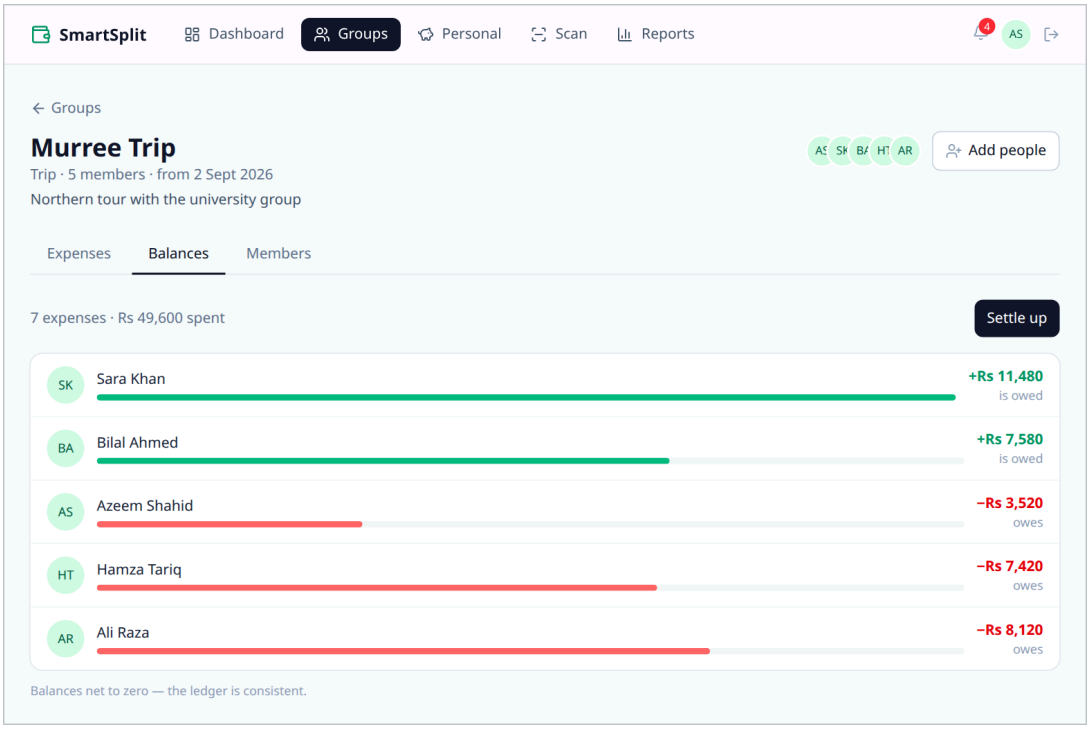



**Figure 5.5:** Group balances, which always net to zero

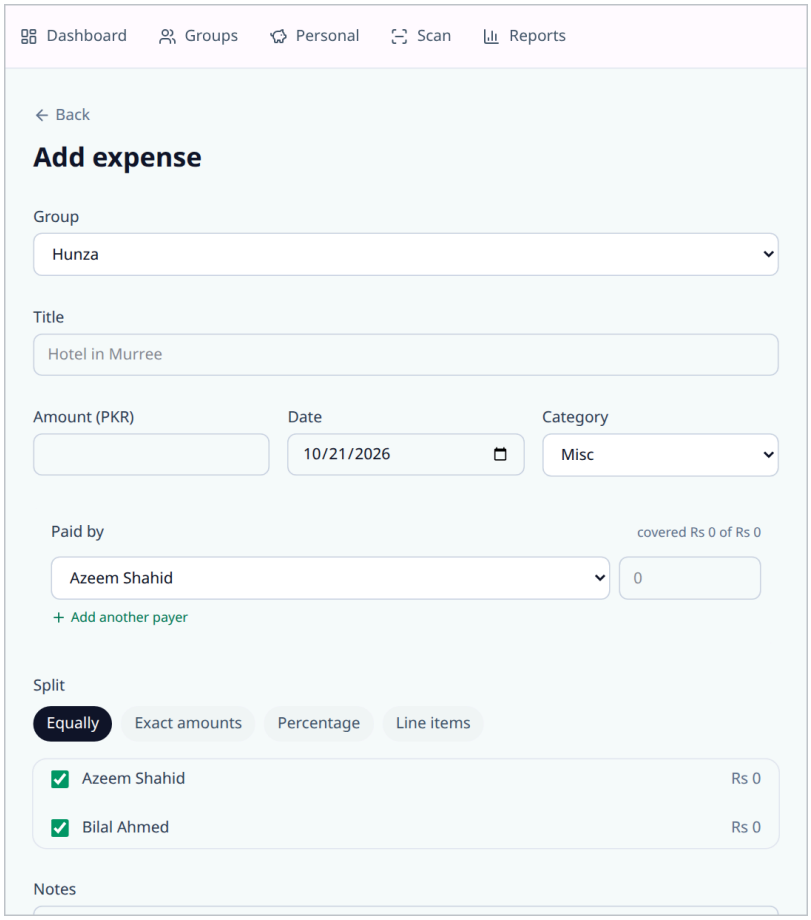



**Figure 5.6:** Add Expense form with multi-payer rows and split methods

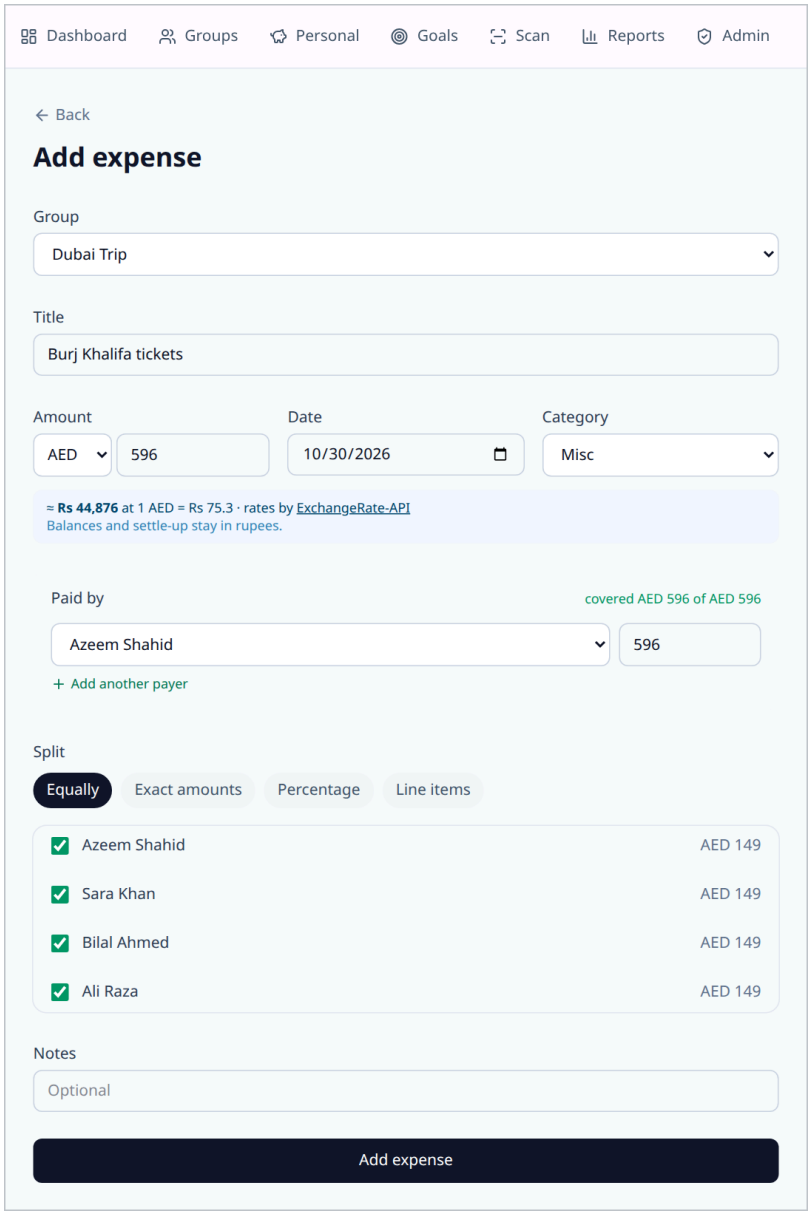



**Figure 5.7:** Expense entered in UAE dirhams with the live rupee preview

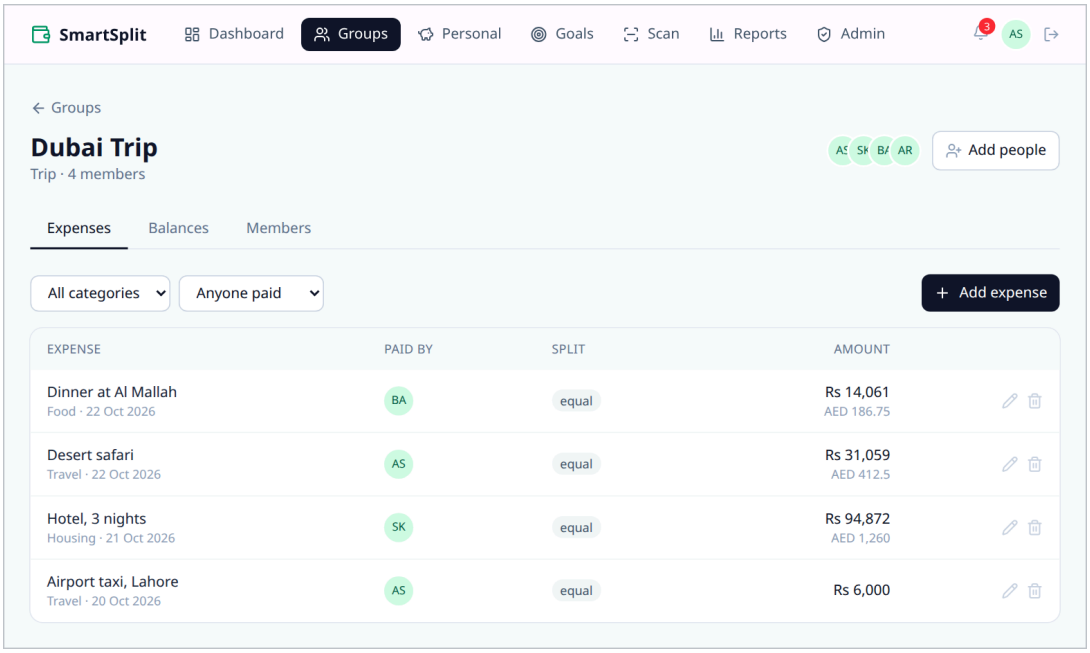



**Figure 5.8:** Dubai Trip expenses: rupee amounts with the original AED figures

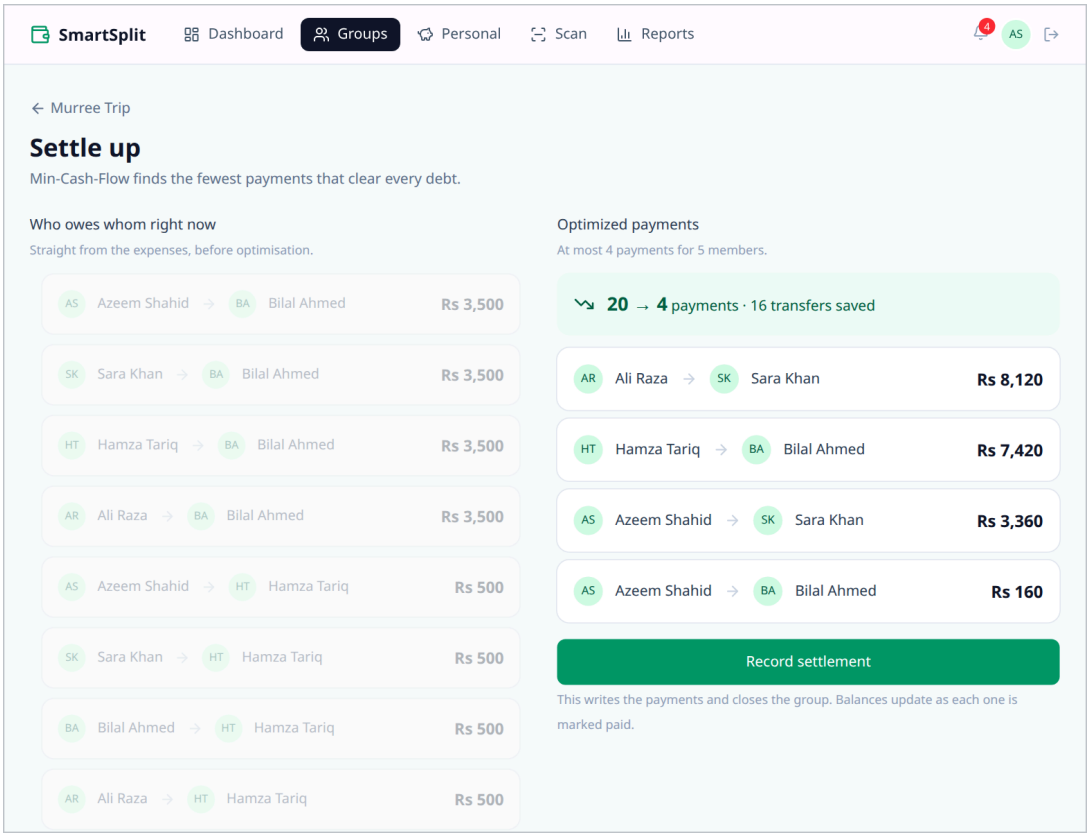



**Figure 5.9:** Settle Up after optimisation: 20 naive debts reduced to 4 payments

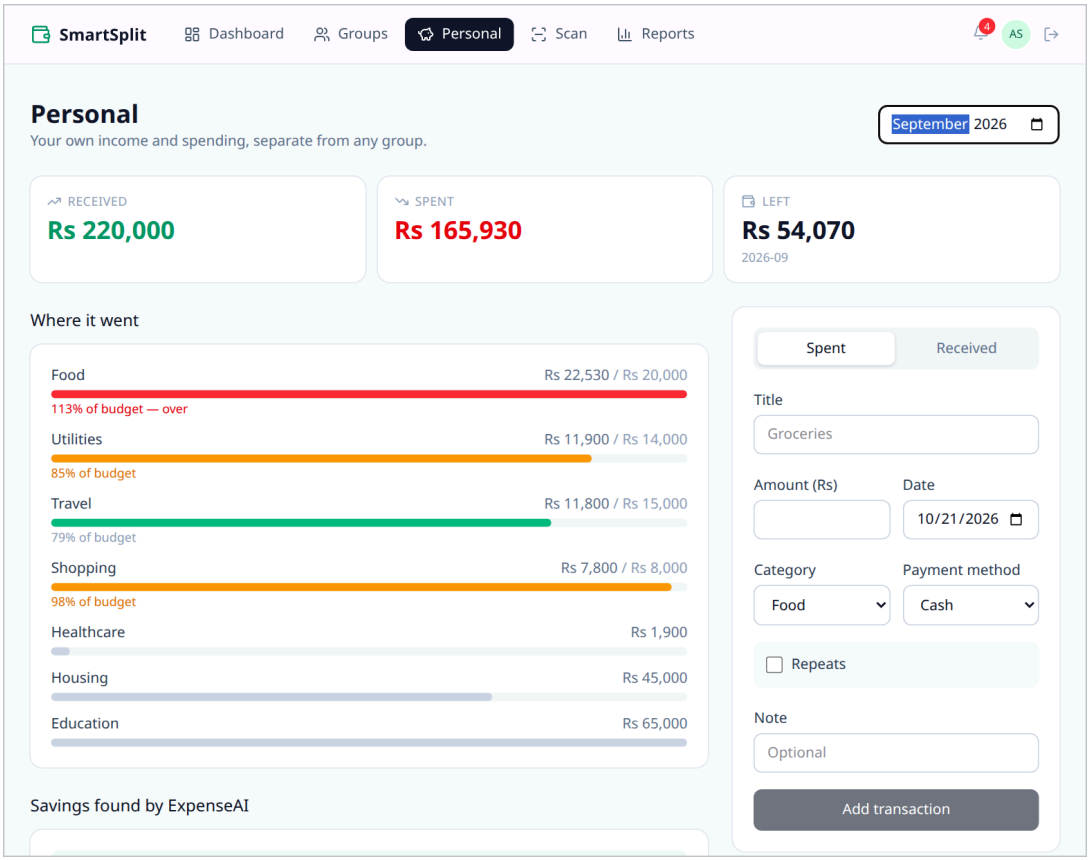



**Figure 5.10:** Personal workspace with budget bands and transaction form

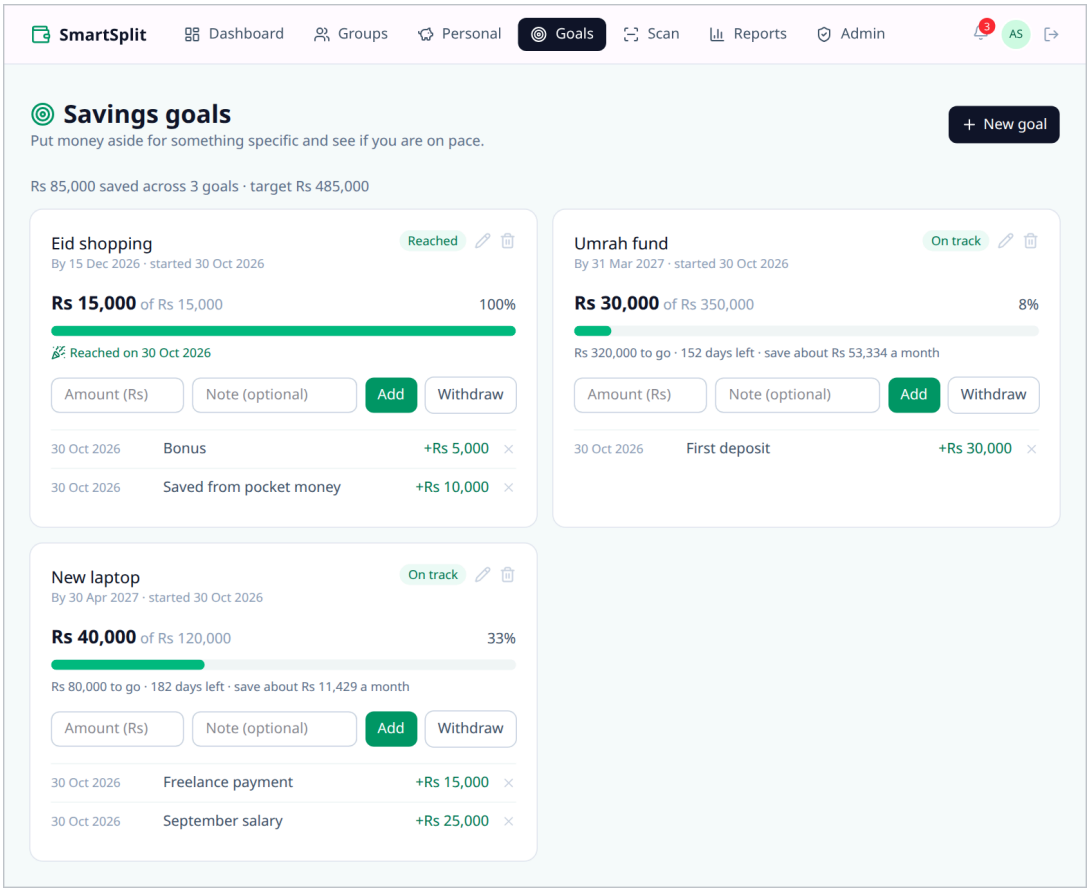



**Figure 5.11:** Savings goals with progress, pace and the monthly amount needed

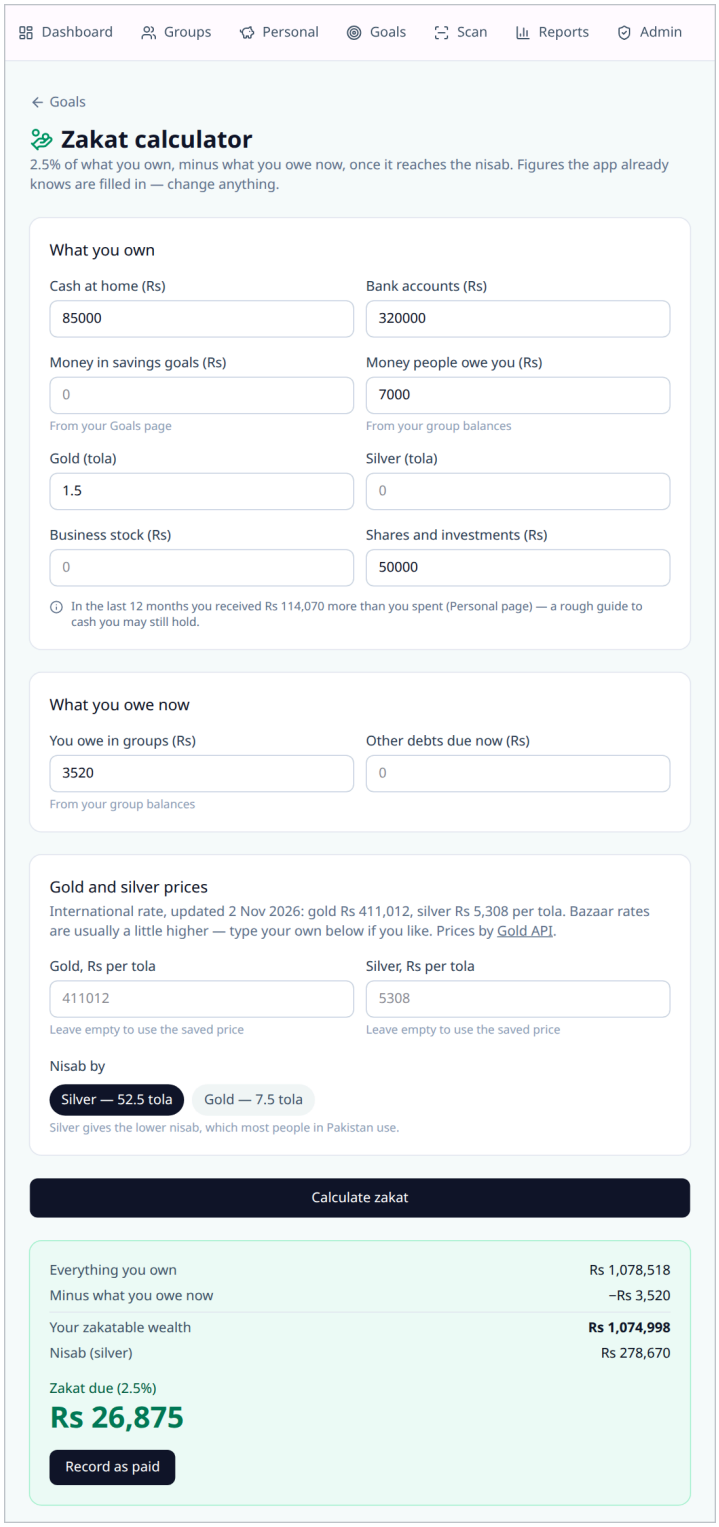



**Figure 5.12:** Zakat calculator with pre-filled figures, automatically fetched metal prices and the result

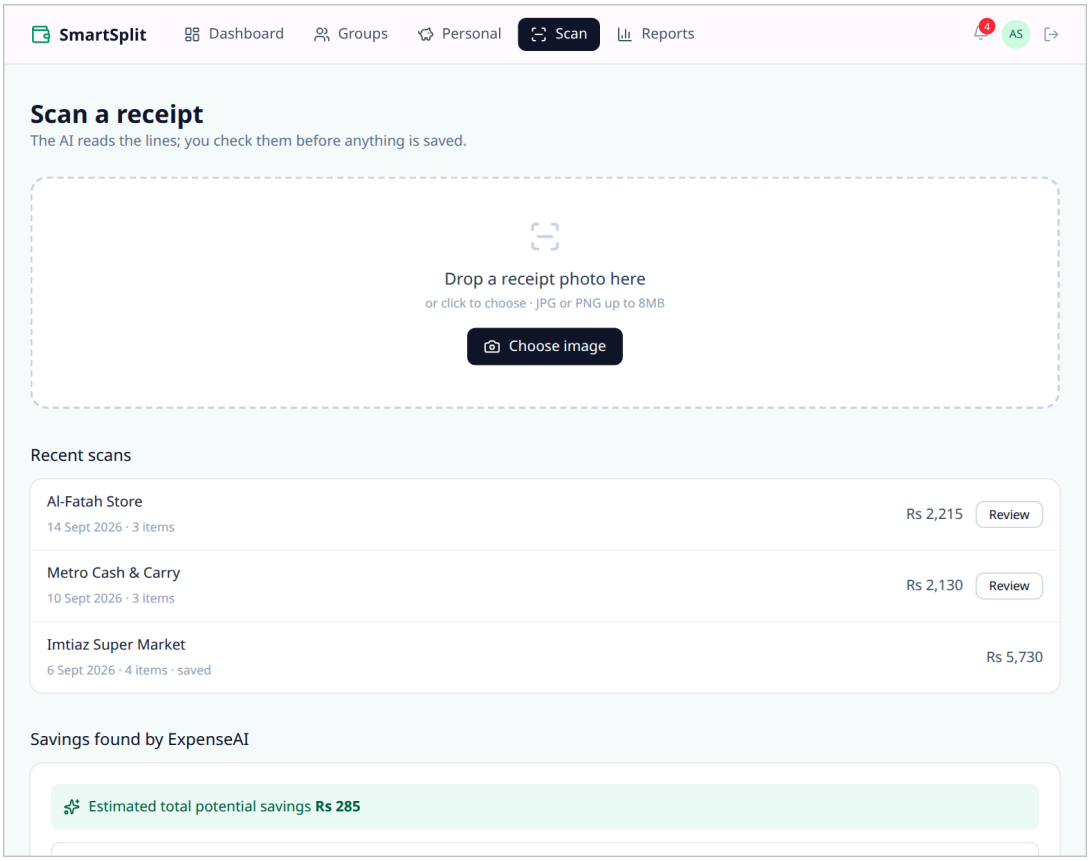



**Figure 5.13:** Receipt scanning page with recent scans and savings found

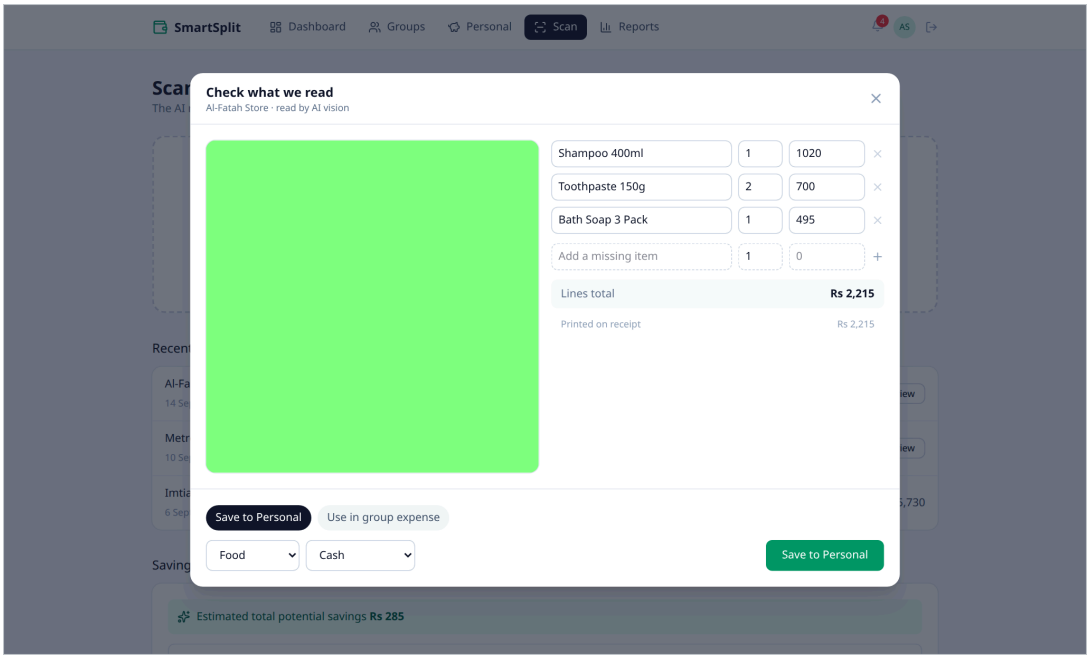



**Figure 5.14:** Human-in-the-loop verification dialog (demonstration data uses a placeholder image)

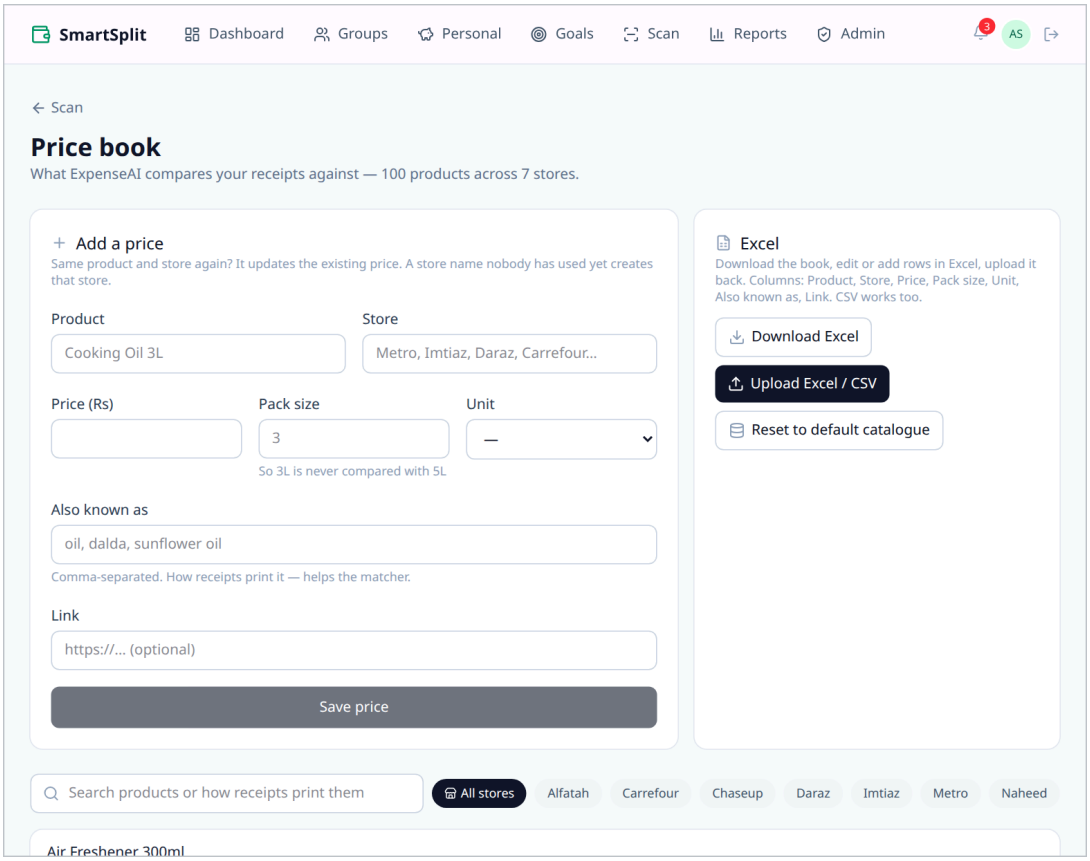



**Figure 5.15:** Price book of benchmark prices, maintained by super administrators

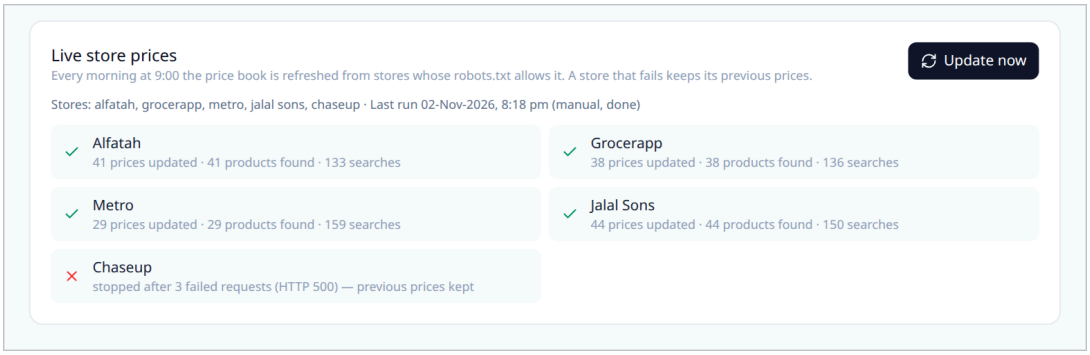



**Figure 5.16:** Live store prices panel on the Admin page after a refresh in which Chase Up failed and kept its previous prices

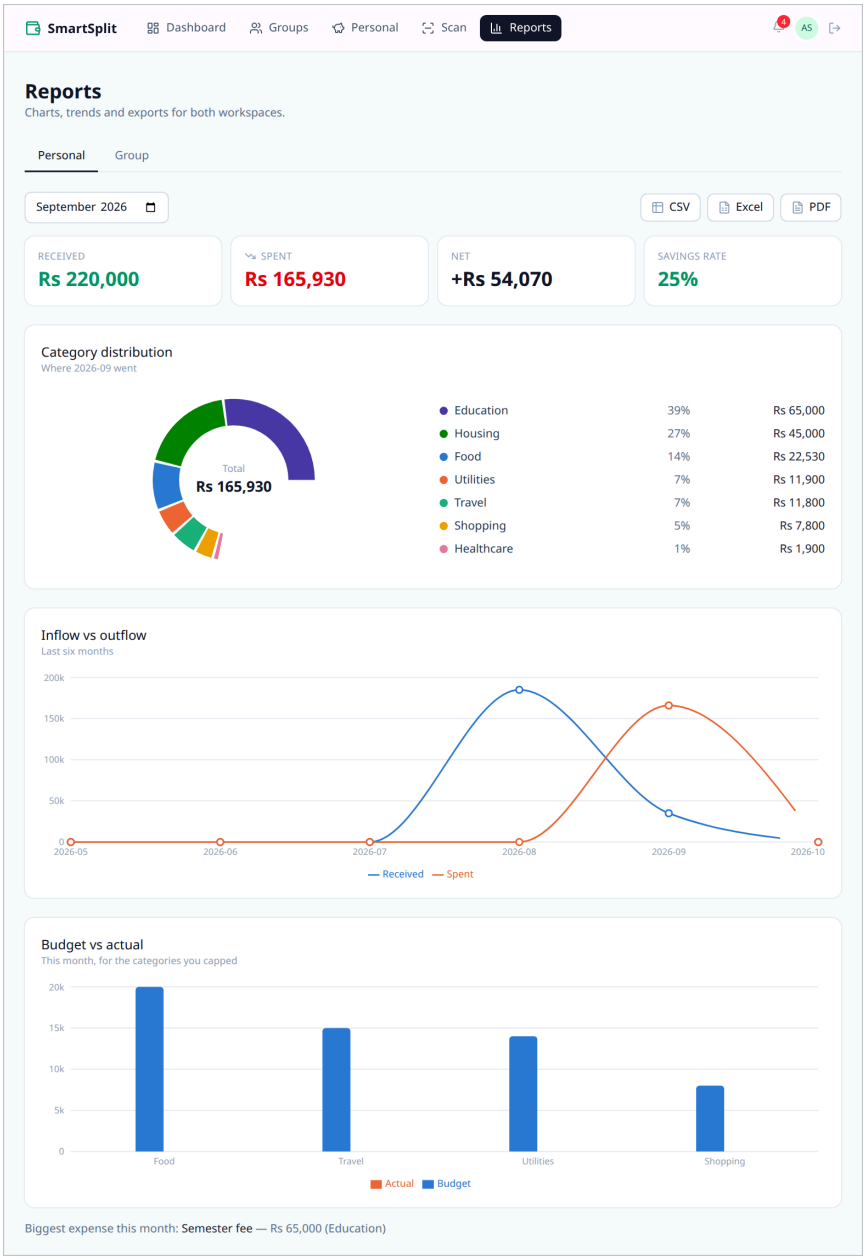



**Figure 5.17:** Personal reports: category distribution, inflow vs outflow, budget vs actual

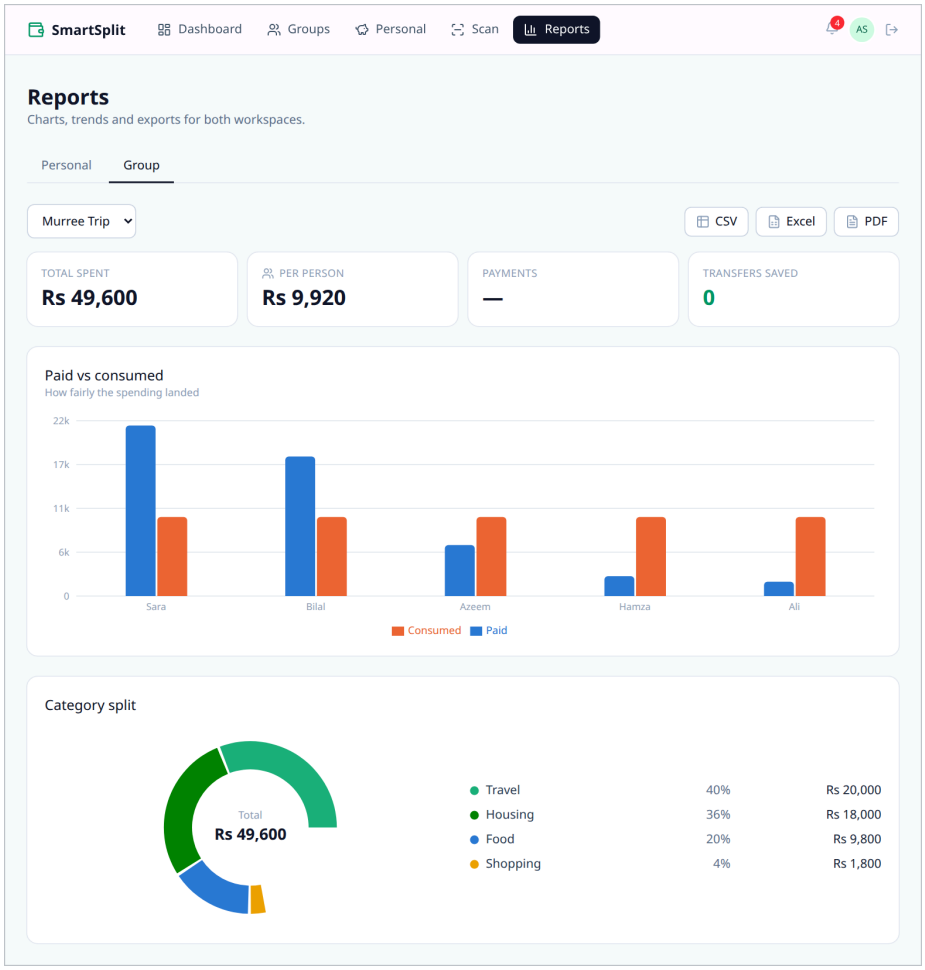



**Figure 5.18:** Group report: paid vs consumed per member and category split

## 5.6 Deployment

The system is live at https://smarsplit.onrender.com. It runs as a single Render web service: the build step installs both packages and builds the React client, and Express serves the built files together with the API. Serving both from one origin keeps the SameSite=Strict refresh cookie first-party and means the browser never makes a cross-origin request; any path outside `/api` returns the client, so deep links such as `/groups/:id` load correctly. The database is a MongoDB Atlas cluster, the health check is `/api/v1/health`, and all secrets are supplied as environment variables.

Deployment is automated and gated by tests. Every push to the main branch on GitHub starts a GitHub Actions workflow that runs the full server test suite and builds the client; Render deploys the new version only when that workflow passes (`autoDeployTrigger: checksPass`), so a failing change never reaches users. On the free tier the service pings its own health route every ten minutes to avoid sleeping during use, and on start-up it loads the default price catalogue if the price book is empty and starts a store-price refresh unless a

recent run has finished or is still going. A seed script (`npm run seed`) creates five demonstration users, two groups including the Murree Trip, twelve personal transactions, budgets, three scanned receipts and the price benchmark.

# CHAPTER 6
# TESTING AND EVALUATION

This chapter describes the testing strategy, summarises the automated test suite, presents selected test cases and the results of security review and defect tracking, and evaluates the settlement engine.

## 6.1 Testing Strategy

- **Unit testing** — pure functions (split calculator, settlement, price matching, zakat calculation, robots.txt rules and store-listing selection, environment guard) are tested in isolation with Vitest.
- **API (integration) testing** — Supertest sends real HTTP requests to the Express application connected to an in-memory MongoDB instance (mongodb-memory-server), verifying status codes, response envelopes, authorisation and database effects. External services (exchange rates, metal prices, online stores and Gmail) are replaced by scripted responses, so no test depends on the internet.
- **Property-based testing** [18] — the settlement engine is tested on hundreds of randomly generated groups to confirm that its guarantees hold for all inputs, not only hand-picked examples. A fixed random seed makes every failure reproducible.
- **Manual and exploratory testing** — every screen was exercised in the browser with the demonstration data, on a desktop and at a 375-pixel phone width, with shared links opened both signed in and signed out; each of the four emails (password reset, invitation on group creation, invitation from the Members tab and join request) was triggered through the user interface and its link followed to the end.

## 6.2 Test Suite Summary

The suite contains 319 tests in 24 files. All 319 tests passed on 2 November 2026 (Table 6.1), and the same suite runs automatically before every deployment.

**Table 6.1:** Automated test results by file

| Test file | Area covered | Tests | Passed |
|---|---|---|---|
| auth.test.js | Register, login, refresh rotation, reuse detection, reset | 19 | 19 |
| profile.test.js | Profile update and password change | 10 | 10 |
| env.test.js | Production environment guard | 7 | 7 |
| health.test.js | Health route and 404 envelope | 2 | 2 |
| admin.test.js | Super admins, overview, default catalogue | 4 | 4 |

| Test file | Area covered | Tests | Passed |
|---|---|---|---|
| groups.test.js | Groups, invites, inbox, member roles, trip currency | 24 | 24 |
| membership.test.js | Direct add, invite, request-and-approve flows | 26 | 26 |
| splitCalculator.test.js | Four split methods and rounding rules | 22 | 22 |
| expenses.test.js | Group expense API, filters, balances | 16 | 16 |
| currency.test.js | Foreign-currency conversion and rounding | 8 | 8 |
| fx.test.js | Exchange rates, fallback, foreign expenses | 12 | 12 |
| settlement.test.js | Min-Cash-Flow algorithm and property tests | 12 | 12 |
| settlementApi.test.js | Preview, record (admin), mark paid, consistency | 11 | 11 |
| personal.test.js | Transactions, budgets, recurring engine, summary | 16 | 16 |
| goals.test.js | Savings goals, contributions, progress | 11 | 11 |
| zakat.test.js | Zakat calculation, metal prices, bazaar price override | 11 | 11 |
| categories.test.js | Built-in and custom categories | 19 | 19 |
| receipts.test.js | Scan pipeline, verification, commit | 18 | 18 |
| ocr.test.js | OCR engines and offline receipt parser | 7 | 7 |
| priceMatch.test.js | Normalisation, similarity, size awareness, fresh prices | 19 | 19 |
| prices.test.js | Price book editing, Excel/CSV import, export | 9 | 9 |
| priceSync.test.js | robots.txt rules, store readers, listing selection, refresh runs | 15 | 15 |
| reports.test.js | Reports, CSV/XLSX/PDF exports, notifications | 19 | 19 |
| mailer.test.js | Email through the Gmail API and its refusal | 2 | 2 |
| **Total** | | **319** | **319** |

## 6.3 Selected Test Cases

**Table 6.2:** Selected test cases

| ID | Test case | Expected result | Result |
|---|---|---|---|
| TC-01 | Split Rs 100 equally among three, payer included | Shares 34 / 33 / 33, payer receives 34 | Pass |
| TC-02 | Exact split totalling Rs 50 less than the bill | 400 with "You're Rs 50 short" | Pass |
| TC-03 | Percentages totalling 90% | 400 with "Percentages add up to 90%, not 100%" | Pass |
| TC-04 | Fractional amount (Rs 10.5) | 400 validation error, nothing stored | Pass |
| TC-05 | Non-member requests group expenses | 403 NOT_A_MEMBER | Pass |

| ID | Test case | Expected result | Result |
|---|---|---|---|
| TC-06 | Plain member attempts to remove a member | 403 NOT_ADMIN | Pass |
| TC-07 | Replay of an already-rotated refresh token | 401 TOKEN_REUSED and all sessions revoked | Pass |
| TC-08 | Forgot password for unknown vs known email | Byte-identical responses | Pass |
| TC-09 | Settle five-member reference group | Exactly 4 payments, all balances zero | Pass |
| TC-10 | 300 random groups of 2–10 members | Balances zero after payments; count ≤ V − 1 | Pass |
| TC-11 | Balances that do not net to zero | Algorithm throws, no payments invented | Pass |
| TC-12 | Second settlement while one is unpaid | 409 SETTLEMENT_IN_PROGRESS | Pass |
| TC-13 | Commit a receipt still processing | Refused; nothing written | Pass |
| TC-14 | Match "Basmati Rice 5kg" against 1 kg benchmark | No match (sizes differ) | Pass |
| TC-15 | Monthly recurring template overdue by three months | Three entries created, schedule advanced | Pass |
| TC-16 | Excel and PDF exports | Valid ZIP (XLSX) and file starting with %PDF | Pass |
| TC-17 | Notifications collection unavailable | Expense creation still succeeds | Pass |
| TC-18 | AED 240 expense at 1 AED = Rs 80 | Stored as Rs 19,200 with the AED figures kept | Pass |
| TC-19 | Three payers of AED 80 each at 1 AED = Rs 75.37 | Payers 6,029 / 6,030 / 6,030, summing to the Rs 18,089 total | Pass |
| TC-20 | Rate provider down, rates saved earlier | Saved rates returned and marked stale | Pass |
| TC-21 | Rate provider down, no rates ever saved | 503 FX_UNAVAILABLE | Pass |
| TC-22 | Withdraw more than a goal holds | 400 GOAL_BELOW_ZERO, goal unchanged | Pass |
| TC-23 | Plain member records a settlement | 403 NOT_ADMIN | Pass |
| TC-24 | Email provider refuses the password-reset email | 200 with the usual message; failure logged | Pass |
| TC-25 | Group share link opened while already signed in | Group preview with an Ask to join button | Pass |
| TC-26 | Every screen at a 375-pixel phone width | No sideways scrolling; all header icons visible | Pass |
| TC-27 | Create a group with trip currency AED, then with "XYZ" | 201 with AED kept; 400 VALIDATION_ERROR | Pass |
| TC-28 | Open Add expense for a new expense in a group whose trip currency is AED | Entry currency starts as AED (checked in the browser) | Pass |
| TC-29 | Zakat on Rs 157,499 with silver at Rs 3,000 per tola | Nisab Rs 157,500; not due; zakat Rs 0 | Pass |

| ID | Test case | Expected result | Result |
|---|---|---|---|
| TC-30 | Zakat on exactly Rs 157,500 at the same price | Due; Rs 3,938 (3,937.5 rounded up) | Pass |
| TC-31 | Plain user saves gold and silver prices | 403 NOT_SUPER_ADMIN | Pass |
| TC-32 | Metal price provider down after one good fetch | Saved prices returned and marked stale | Pass |
| TC-33 | Store whose robots.txt disallows its search path | Store skipped with the reason recorded; search never called; prices unchanged | Pass |
| TC-34 | Store answering HTTP 503 to every request | Stopped after 3 failed requests; previous prices kept; run completes | Pass |
| TC-35 | Listings for engine oil 3 L, cooking oil without a size, and cooking oil at Rs 99 or Rs 3,200 | None accepted for "Cooking Oil 3L" | Pass |
| TC-36 | Three 3-litre cooking oils at Rs 1,645, 1,890 and 1,930, and a 5-litre can | Rs 1,890 listing (middle price) chosen | Pass |
| TC-37 | Sitemap lists a cooking-oil page and a tea page; only oil is in the book | Only the oil page is opened and its price stored | Pass |
| TC-38 | Start-up while a refresh begun an hour earlier is still running | No second run is started | Pass |
| TC-39 | Gmail refuses a message (daily quota exceeded) | Sending fails with Google's reason, so the caller can fall back to the link | Pass |
| TC-40 | Sugar 1 kg priced Rs 158 by a store today and Rs 120 in a 30-day-old row | Rs 158 compared; the old row counts again only when no price is recent | Pass |

## 6.4 Property-Based Testing of Settlement

The strongest evidence for the settlement engine comes from property-based tests. In each of 300 rounds, a random group of 2 to 10 members and 1 to 6 expenses is generated through the real split calculator; 100 further rounds use multi-payer expenses. For every round the test asserts three properties: (1) applying the returned payments brings every balance to exactly zero; (2) the number of payments is at most V − 1; and (3) every payment amount is a positive integer. All 400 rounds pass.

## 6.5 Security Testing

All 38 routes that existed at the time of the security review were audited for authentication, validation and ownership checks. Four gaps were found and fixed (Table 6.3). A dependency audit (`npm audit`) reported zero vulnerabilities after a transitive dependency was pinned. The rate limits were later raised (3,000 requests per 15 minutes overall and 60 sign-ins per 15 minutes per address) so that a whole classroom sharing one public IP address can use the system at once; bcrypt at cost 12 still makes password guessing slow. The five endpoints

added later for zakat and the price refresh follow the same pattern: all require authentication, saving metal prices and both price-refresh routes also require the super-admin role, and the two request bodies (a zakat calculation and the metal prices) are validated with Zod.

**Table 6.3:** Security review findings and fixes

| Gap | Risk | Fix |
|---|---|---|
| No global rate limit | Any authenticated endpoint could be flooded | Global limiter: 600 requests per 15 minutes |
| Unlimited receipt scans | Each scan calls a paid AI service | Scan limiter: 30 per hour |
| Permanent public image URLs | A leaked receipt link would work forever | Authenticated uploads; signed URLs valid for 1 hour |
| Proxy not trusted | All users behind the host proxy shared one rate-limit bucket | trust proxy enabled |

## 6.6 Defects Found and Fixed

**Table 6.4:** Notable defects found during testing

| Defect | Cause and fix |
|---|---|
| Phantom saving of Rs 1,470 on rice | Normalisation removed pack sizes, so 5 kg matched 1 kg. Fixed by parsing sizes separately and requiring them to agree. |
| Title-only edit wiped an expense split | Partial validation re-applied default values. Fixed by rebuilding missing split input from the stored splits. |
| Background price match clashed with user edits | Load-modify-save caused a version conflict. Fixed with an atomic update. |
| Password account reported as having no password | The password hash was excluded from the query. Fixed with a response transform instead. |
| Unmatched routes returned 401 instead of 404 | Router-level authentication ran before the 404 handler. Fixed by applying authentication per route. |
| Personal page failed to render with a budget cap | A variable missed during a renaming refactor. Fixed and verified in the browser. |
| Blank page right after a deployment | Open tabs requested old, renamed code files. Fixed with an error boundary that reloads once when a code file is missing. |
| Offline OCR could stop the server | Tesseract.js downloaded language data at run time and a failed download crashed the process. Fixed by bundling the data and handling worker errors. |
| Invitation emails silently not delivered | The email provider refused messages until the account was activated, and mail errors were hidden. Fixed by logging every send and warning the admin, with WhatsApp and copy-link as alternatives. |
| Empty price book on a new database | Benchmark prices were only loaded by the seed script. Fixed by loading the default catalogue on start-up when the collection is empty. |
| Share link crashed for signed-in users | For one render after start-up the page still held the empty signed-out result. Fixed by waiting for the group preview before drawing the page. |

| Defect | Cause and fix |
|---|---|
| Forgot password failed with a server error | When the email provider refused the reset email, the error reached the user. Fixed by logging the failure and giving the usual answer, which also keeps account existence private. |
| Pages wider than a phone screen | At 375 pixels the administrator header, the Personal transactions table and the Admin services list pushed the page sideways. Fixed by letting grid columns shrink and fitting the header icons. |
| Rupee amounts labelled as dirhams | A group's currency was a free-text box that only relabelled rupee amounts, so Rs 31,059 appeared as "AED 31,059", and any three letters were accepted. Fixed by turning it into a trip currency chosen from the supported list that sets the starting currency of new expenses, and labelling every rupee figure "Rs". |
| Live prices suggesting savings that did not exist | Taking the cheapest matching store listing often picked a different grade, such as broken basmati rice. Fixed by taking the middle-priced listing, within 0.6 to 1.6 times the price book's median. |

## 6.7 Evaluation

Table 6.5 evaluates the system against its objectives.

**Table 6.5:** Evaluation against objectives

| Objective | Evidence | Status |
|---|---|---|
| Personal workspace with budgets and recurring entries | FR-05 to FR-10 implemented; 16 personal tests pass | Achieved |
| AI receipt scanning with fallback and verification | FR-11 to FR-14; 18 receipt tests pass | Achieved |
| Savings suggestions from benchmark prices | FR-15 and FR-34; 100 products, 609 indicative prices; a live refresh updated 191 prices from five stores within robots.txt; 19 matching and 14 price-refresh tests pass | Achieved |
| Group workspace with four exact split methods | FR-16 to FR-20; 22 calculator and 16 expense tests pass | Achieved |
| Settlement in at most V − 1 payments | 400 property-test rounds; Murree Trip 20 → 4 payments | Achieved |
| Savings goals, zakat and multi-currency trips | FR-27, FR-28 and FR-31 to FR-33; 42 currency, rate, goal and zakat tests pass; AED 412.50 = Rs 31,059 verified in the browser | Achieved |
| Reports and exports | FR-24 and FR-25; CSV, XLSX and PDF verified | Achieved |
| Secure authentication and authorisation | Token rotation, reuse detection, membership checks, audit | Achieved |
| Automated verification | 319 of 319 tests passing | Achieved |
| Public, test-gated deployment | Live on Render with MongoDB Atlas; GitHub Actions must pass before each deploy | Achieved |

# CHAPTER 7
# CONCLUSION AND FUTURE WORK

## 7.1 Conclusion

This project delivered SmartSplit & ExpenseAI, a secure web application that brings personal finance management and group expense settlement together in one account for Pakistani users. Receipt photos are converted into verified transactions with the help of a vision language model, verified items are compared with local market prices to suggest savings, and group debts are settled by a Min-Cash-Flow engine that never needs more than $V - 1$ payments. Savings goals help users plan ahead, a zakat calculator estimates what is due from the user's own figures and current metal prices, trips abroad can be recorded in the currency on the bill, the price book is refreshed every morning from five online stores within the limits of their robots.txt, and the system is publicly deployed with every change tested before release.

Three design decisions proved central to the system's reliability: storing money as integer rupees with a single rounding rule, recomputing balances from the underlying expenses instead of storing them, and keeping a human in the loop for every AI-extracted value. Together with server-side authorisation and a comprehensive automated test suite of 319 passing tests, including property-based tests of the settlement guarantees, they give strong confidence that every rupee in the system is accounted for exactly.

## 7.2 Limitations

- The greedy settlement guarantees at most $V - 1$ payments but is not always the absolute minimum, which is an NP-hard problem.
- Live prices come from only five online stores, and only for products they list under the same name and pack size; the remaining prices in the book are indicative figures. A change to a store's website can stop its prices updating until its reader is adapted, and a store whose robots.txt comes to disallow a path it needs is skipped; in both cases its previous prices are kept.
- Overlapping price refreshes are prevented by an in-process lock, with the start-up check also reading recent runs from the database; if the API were ever run on several instances, the scheduled and manual refreshes would need a lock shared through the database.
- The zakat calculator gives an estimate: it uses the international metal price unless a bazaar price has been entered, and scholars differ on how some assets are treated.
- The application records payments but does not transfer money itself.
- OCR accuracy depends on photo quality; poor images require more manual correction.

- A single typo in a short product name can fall below the 0.8 similarity threshold; common misspellings are handled through aliases instead.
- On free hosting tiers the server can still sleep when nobody uses it for a long time, so the first request after a pause is slow and the nightly jobs may run late.
- Exchange rates are refreshed once a day from a free provider, so a converted expense uses that day's market rate rather than the exact rate charged by the user's bank or card.
- Email delivery depends on an external provider. As of 2 November 2026 Brevo had still not activated the account for sending; sending through the Gmail API is implemented and tested but works only once its three keys are set on the server. Until then invitations are shared by link or WhatsApp and password-reset emails do not arrive.

## 7.3 Future Work

1. Integration with JazzCash, Easypaisa and Raast payment APIs so that a settlement payment can be made directly from the app.
2. A native mobile application reusing the existing REST API.
3. Automatic category suggestions using machine learning on the user's past transactions.
4. Automatic weekly import of official prices from the Pakistan Bureau of Statistics Sensitive Price Indicator (SPI) into the price book.
5. Letting the user enter the exact rate their card or bank charged for a foreign-currency expense.
6. An exact minimum-payment mode for small groups, using subset search, offered alongside the greedy method.

# REFERENCES

[1] T. Verhoeff, "Settling multiple debts efficiently: An invitation to computing science," *Informatics in Education*, vol. 3, no. 1, pp. 105–126, 2004.

[2] M. R. Garey and D. S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*. New York, NY, USA: W. H. Freeman, 1979.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 4th ed. Cambridge, MA, USA: MIT Press, 2022.

[4] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th Int. Conf. Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[5] L. R. Dice, "Measures of the amount of ecologic association between species," *Ecology*, vol. 26, no. 3, pp. 297–302, 1945.

[6] M. Jones, J. Bradley, and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[7] D. Hardt, Ed., "The OAuth 2.0 authorization framework," IETF RFC 6749, Oct. 2012.

[8] N. Provos and D. Mazières, "A future-adaptable password scheme," in *Proc. USENIX Annual Technical Conference, FREENIX Track*, 1999.

[9] OWASP Foundation, "OWASP Top 10:2021." [Online]. Available: https://owasp.org/Top10/

[10] D. Goldberg, "What every computer scientist should know about floating-point arithmetic," *ACM Computing Surveys*, vol. 23, no. 1, pp. 5–48, 1991.

[11] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, Univ. of California, Irvine, 2000.

[12] MongoDB, Inc., "Aggregation operations," MongoDB Manual. [Online]. Available: https://www.mongodb.com/docs/manual/aggregation/

[13] Meta Platforms, Inc., "React documentation." [Online]. Available: https://react.dev/

[14] Anthropic, "Claude API documentation: vision and structured outputs." [Online]. Available: https://docs.anthropic.com/

[15] Splitwise, Inc., "Splitwise." [Online]. Available: https://www.splitwise.com/

[16] Tricount, "Tricount." [Online]. Available: https://www.tricount.com/

[17] BudgetBakers, "Wallet." [Online]. Available: https://budgetbakers.com/

[18] K. Claessen and J. Hughes, "QuickCheck: A lightweight tool for random testing of Haskell programs," in *Proc. 5th ACM SIGPLAN Int. Conf. Functional Programming (ICFP)*, 2000, pp. 268–279.

[19] State Bank of Pakistan, "Raast: Pakistan's instant payment system." [Online]. Available: https://www.sbp.org.pk/

[20] I. Sommerville, *Software Engineering*, 10th ed. Harlow, U.K.: Pearson, 2016.

[21] Mongoose, "Mongoose ODM documentation." [Online]. Available: https://mongoosejs.com/docs/

[22] ExchangeRate-API, "Free exchange rate API (open access)." [Online]. Available: https://www.exchangerate-api.com/docs/free

[23] M. Koster, G. Illyes, H. Zeller, and L. Sassman, "Robots Exclusion Protocol," IETF RFC 9309, Sep. 2022.

[24] Schema.org, "Product," schema.org vocabulary. [Online]. Available: https://schema.org/Product

[25] Gold API, "Free real-time gold and silver price API." [Online]. Available: https://gold-api.com/

[26] Google, "Gmail API: Sending email," Google for Developers. [Online]. Available: https://developers.google.com/gmail/api/guides/sending

# APPENDIX A: API ENDPOINT REFERENCE

All endpoints are prefixed with `/api/v1`. Every endpoint except registration, sign-in, token refresh, Google sign-in, password reset and the health check requires a valid access token; group endpoints additionally require active membership, and admin endpoints the admin role.

**Table A.1:** REST API endpoints

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Auth | POST | `/auth/register` | Create an account |
| Auth | POST | `/auth/login` | Sign in with email and password |
| Auth | POST | `/auth/refresh` | Rotate refresh token, issue access token |
| Auth | POST | `/auth/logout` | Revoke the current session |
| Auth | GET | `/auth/me` | Current user |
| Auth | PATCH | `/auth/me` | Update profile |
| Auth | POST | `/auth/change-password` | Change password |
| Auth | POST | `/auth/google` | Google sign-in |
| Auth | POST | `/auth/forgot-password` | Email a reset link |
| Auth | POST | `/auth/reset-password/:token` | Set a new password |
| Users | GET | `/users/search` | Find users to add to a group |
| Groups | POST | `/groups` | Create a group |
| Groups | GET | `/groups` | List my groups with totals |

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Groups | GET | `/groups/invites` | My invitations and join requests |
| Groups | POST | `/groups/:id/invites/accept` | Accept an invitation |
| Groups | DELETE | `/groups/:id/invites` | Decline an invitation or withdraw a request |
| Groups | PATCH | `/groups/:id/members/:memberId` | Promote or demote a member (admin) |
| Groups | GET | `/groups/:id` | Group details (member) |
| Groups | PATCH | `/groups/:id` | Update group (admin) |
| Groups | POST | `/groups/:id/members` | Add or invite a member (admin) |
| Groups | DELETE | `/groups/:id/members/:userId` | Remove a member (admin) |
| Groups | PATCH | `/groups/:id/members/:memberId/approve` | Approve a join request (admin) |
| Groups | POST | `/groups/:id/members/:memberId/resend` | Resend an invite (admin) |
| Groups | POST | `/groups/:id/join-code/rotate` | Rotate the share code (admin) |
| Groups | GET | `/join/:inviteToken` | Preview an invite link |
| Groups | POST | `/join/:inviteToken` | Accept an invite link |
| Groups | GET | `/join-request/:joinCode` | Preview a group by share code |
| Groups | POST | `/join-request/:joinCode` | Request to join |

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Expenses | POST | `/groups/:id/expenses` | Add a group expense |
| Expenses | GET | `/groups/:id/expenses` | List expenses (filters, pages) |
| Expenses | GET | `/groups/:id/balances` | Net balances by aggregation |
| Expenses | GET | `/expenses/:expenseId` | One expense |
| Expenses | PATCH | `/expenses/:expenseId` | Edit an expense |
| Expenses | DELETE | `/expenses/:expenseId` | Soft-delete an expense |
| Settlements | POST | `/groups/:id/settle/preview` | Compute payments (no write) |
| Settlements | POST | `/groups/:id/settle` | Record a settlement (admin) |
| Settlements | GET | `/groups/:id/settlements` | Payments and audit trail |
| Settlements | PATCH | `/settlements/:settlementId/pay` | Mark a payment paid |
| Personal | POST | `/transactions` | Add a transaction |
| Personal | GET | `/transactions` | List transactions (filters) |
| Personal | PATCH | `/transactions/:transactionId` | Edit a transaction |
| Personal | DELETE | `/transactions/:transactionId` | Delete a transaction |
| Personal | GET | `/personal/summary` | Monthly summary |
| Personal | GET | `/budgets` | List budgets |
| Personal | PUT | `/budgets` | Create or update a budget |

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Personal | DELETE | `/budgets/:budgetId` | Delete a budget |
| Goals | GET | `/goals` | List my goals with progress |
| Goals | POST | `/goals` | Create a goal |
| Goals | PATCH | `/goals/:goalId` | Edit a goal |
| Goals | DELETE | `/goals/:goalId` | Delete a goal |
| Goals | POST | `/goals/:goalId/contributions` | Deposit or withdraw |
| Goals | DELETE | `/goals/:goalId/contributions/:contributionId` | Remove a contribution |
| Categories | GET | `/categories` | Built-in and custom categories |
| Categories | POST | `/categories` | Add a custom category |
| Categories | GET | `/categories/:categoryId/usage` | Usage before deletion |
| Categories | DELETE | `/categories/:categoryId` | Delete a custom category |
| Receipts | POST | `/receipts/scan` | Upload a receipt (202) |
| Receipts | GET | `/receipts` | List receipts |
| Receipts | GET | `/receipts/:receiptId` | Receipt with OCR result |
| Receipts | POST | `/receipts/:receiptId/items` | Add a line |
| Receipts | PATCH | `/receipts/:receiptId/items/:index` | Edit a line |
| Receipts | DELETE | `/receipts/:receiptId/items/:index` | Remove a line |
| Receipts | POST | `/receipts/:receiptId/commit` | Save to personal or group draft |
| Reports | GET | `/reports/personal/monthly` | Monthly personal report |

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Reports | GET | `/reports/personal/trends` | Six-month trends |
| Reports | GET | `/reports/groups/:id/summary` | Group summary report |
| Reports | GET | `/reports/export` | CSV, XLSX or PDF export |
| Notifications | GET | `/notifications` | Notification feed |
| Notifications | PATCH | `/notifications/:notificationId/read` | Mark one read |
| Notifications | PATCH | `/notifications/read-all` | Mark all read |
| Exchange rates | GET | `/fx/rates` | Rupee rates for supported currencies |
| Price book | GET | `/prices` | Benchmark prices by product and store |
| Price book | GET | `/prices/export` | Download as Excel |
| Price book | POST | `/prices` | Add or update a price (super admin) |
| Price book | DELETE | `/prices/:priceId` | Delete a price (super admin) |
| Price book | POST | `/prices/import` | Import Excel or CSV (super admin) |
| Price book | POST | `/prices/seed` | Load the default catalogue (super admin) |
| Zakat | GET | `/zakat` | Pre-filled figures and gold and silver prices |

| Resource | Method | Path | Purpose |
|---|---|---|---|
| Zakat | POST | `/zakat/calculate` | Work out the zakat due |
| Zakat | PUT | `/zakat/rates` | Save bazaar gold and silver prices (super admin) |
| Admin | GET | `/admin/overview` | Totals and newest users and groups (super admin) |
| Admin | GET | `/admin/price-sync` | Stores and the last ten price refreshes (super admin) |
| Admin | POST | `/admin/price-sync` | Start a store-price refresh (super admin, 202) |
| System | GET | `/health` | Health and database status |

# APPENDIX B: USER MANUAL

This appendix explains how to perform the main tasks in the application.

**Table B.1:** Step-by-step guide to the main tasks

| Task | Steps |
|---|---|
| Create an account | Open the website and select "Create an account", or use "Sign in with Google". Enter name, email and a password of at least 8 characters. |
| Record a transaction | Open Personal. In the form, choose Spent or Received, enter title, amount, date, category and payment method, optionally tick "Repeats" for a weekly or monthly bill, and select "Add transaction". |
| Set a budget | On the Personal page, open the budgets editor, choose a category and enter a monthly cap. Bars turn amber at 80% and red at 100%. |
| Scan a receipt | Open Scan and drop a photo or select "Choose image". When reading finishes, check every line in the dialog, correct anything wrong, then select "Save to Personal" or "Use in group expense". |
| Create a group | Open Groups and select "New group". Enter a name, type, trip currency and start date, and add members by email. |
| Join a group | Open an invite link received by email or WhatsApp, accept an invitation from the inbox on the Groups page, or enter a share code and wait for an admin to approve the request. |
| Add a shared expense | In a group, select "Add expense". Enter title and amount, choose who paid (add more payers if needed), choose a split method and fill in the shares. Save becomes available when everything adds up. |
| Add an expense abroad | In Add expense, the currency next to the amount starts as the group's trip currency; change it if the bill is in another currency (for example AED) and type the amount from the bill. The form shows the rupee amount and the rate; the expense is saved in rupees. |
| Settle up | In a group, open Balances and select "Settle up", then "Simplify with Min-Cash-Flow". Review the payments; a group admin then selects "Record settlement". After paying through JazzCash, Easypaisa, Raast, bank or cash, select "Mark paid". |
| Save for a goal | Open Goals and select "New goal". Enter what you are saving for, the target amount and the date. Use Add or Withdraw on the goal card whenever you put money aside or take some out. |
| Calculate zakat | Open Goals and select "Zakat calculator". Check the figures filled in from your goals and group balances, add cash, bank balances, gold, silver and other assets and any debts due now, choose the silver or gold nisab, and select "Calculate zakat". If zakat is due, "Record as paid" adds it to Personal after you confirm. |
| Refresh store prices (admin) | Open Admin. The "Live store prices" panel lists the stores and the result of the last refresh for each one. Select "Update now" to refresh at once; the refresh takes a few minutes and continues in the background. |
| Reports and exports | Open Reports, choose the Personal or Group tab and the month, and use the CSV, Excel or PDF buttons to download. |

# APPENDIX C: INSTALLATION GUIDE

The following commands install and run the system locally. Node.js 20 or later and a MongoDB instance (local or Atlas) are required.

```
git clone <repository-url> smartsplit && cd smartsplit
npm run install:all
cp server/.env.example server/.env      # set MONGO_URI, JWT secrets, API keys
cp client/.env.example client/.env      # set VITE_API_URL
npm run seed                            # optional: demo data (erases the
database)
npm run dev                             # API on :5000, client on :5173
npm test                                # run the 319 automated tests
npm --prefix server run prices:sync     # optional: refresh store prices now
npm --prefix server run gmail:token     # optional: one-time Gmail API token
```

Required environment variables on the server are `MONGO_URI`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET` and `CLIENT_URL`. Optional variables enable Google sign-in (`GOOGLE_CLIENT_ID`), the vision model (`ANTHROPIC_API_KEY`), Cloudinary (`CLOUDINARY_`*) and email (`GMAIL_CLIENT_ID`, `GMAIL_CLIENT_SECRET` and `GMAIL_REFRESH_TOKEN` for the Gmail API, or `SMTP_`*); without them the system falls back to Tesseract.js, inline images and console-printed email links, so its core features run without these services. `gmail:token` needs `GMAIL_CLIENT_ID` and `GMAIL_CLIENT_SECRET` set and prints the refresh token; `prices:sync` reads the online stores, so it needs an internet connection.